Introduction to Programming

W01 Introduction

Stephan Krusche

15 October 2025

Technical University of Munich

Welcome to
Introduction to Programming (ITP)

Teaching concept: AI enhanced interactive learning

- **Synchronous** 🔄

  No traditional lecture!

  - Onsite workshops with exercises (+ livestream)
  - Onsite tutorial groups with exercises and presentations

  No lecture either! **Practice**!

- **Asynchronous** 🕒
  - Recordings, slides, and readings
  - Homework for self-study
- **Great support** 🤗 3 instructors and 39 tutors for ~800 students
- Digital learning tools with Artemis and many different activities
- AI support for personalized help with Iris

→ **Best possible learning experience** 🌟🎓 for everyone

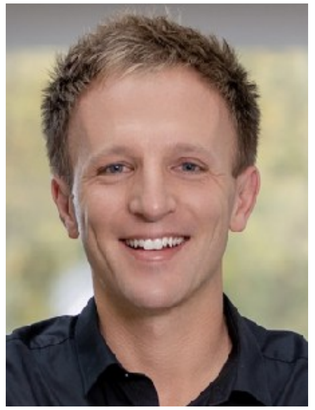
Expectations

- Be **open** to new technologies, learning methods and teaching approaches
- Be **active** and **curious**, learn to **organize** yourself
- Be **kind** and **patient**
- **Bear with us** if something does not work out as expected

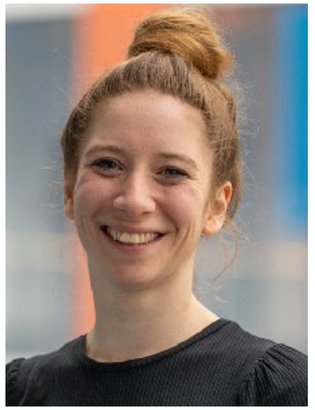
Teaching goals

- You improve your **problem solving** and **soft skills**
- You understand **programming concepts** and learn **how to program**
- You achieve the **best possible grade** in the course

Course team



Stephan (Krusche)
Main lecturer



Ramona (Beinstingel)
Exercise instructor

Markus (Paulsen)
Exercise instructor

Aniruddh (Zaveri)
Experienced tutor

Simon (Bohnen)
Experienced tutor

Jana (Friedrich)
Experienced tutor

Konrad (Weiß)
Experienced tutor

Phoebe (Morrien)
Experienced tutor

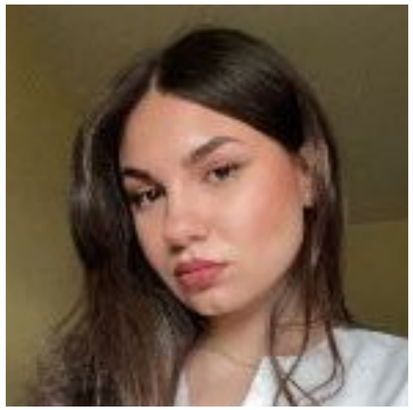
Liam (Berger)
Experienced tutor

Sindi (Buklaji)
Experienced tutor

Course team (tutors)

Tuguldur Batkhishig

Nico Noell



Maia Filip

Maximilian Gutke

Lana Atiya

Samhitha Girish Jois

Daniil Gavlitskii

Beyza Tuba Ulas

Fuat Sekban

Güray Kaleli

Course team (tutors)

Hanna Lib

Huu Khiem Nguyen

Kevin Fischer

Atharva Pradeep Mathapati

Olaf Taczkowski

Mihai Baran

Eric Iambor

Florian Briksa

Catherine Kalra

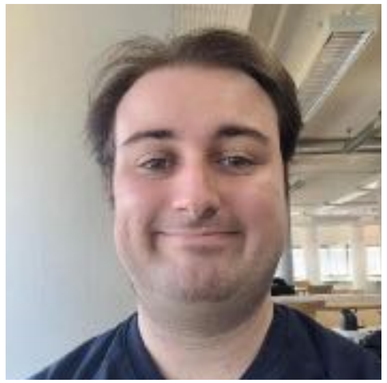
Nadia Çurumi

Course team (tutors)

Valentin Herrmann

Mariya Koeva

Deniz Öztürk



Kristi Balla

Kiara Pia Copony

Arnav

Aleksandra Topalova

Ilgın Taşeli

Melissa Sioukri Oglou

Hannes Harbeck

Fabian Böhm

Firat Akbaba

Course setup 🌟

- **Workshop** 📝
  - Onsite workshops with in-class exercises (+ livestream & recordings)
  - 4 x 40min units (with breaks in-between)
  - Focus on interaction and activation (**no** traditional lecture!)
  - Digital learning tools to create the **best possible learning experience**
- **Tutorial groups** 👥
  - 3 hour slots (with small breaks) in small onsite tutorial groups (~15 students) with intense supervision
  - Focus on group exercises and student presentations (**no** lecture!)
- **Homework** 🛠️
  - Weekly homework exercises which must be solved independently
- **Assessment** 📊
  - Two supervised exercises (computer-based), presentations, a team project, and a bonus

Requirements for this course

- **Assumptions**
  - You want to learn how to program
  - You want to learn how to solve problems
- **Prerequisites**
  - You have your own laptop and use it during the workshop and exercises!
- **Beneficial**
  - You have practical experience with programming
  - You have already participated in a software project

Main learning goals

- **Understand** and **apply** the fundamental concepts of computer science at a basic, practical, but scientific level: algorithms, syntax, semantics, efficiency
- **Solve** algorithmic **problems**
- **Program** simple applications in the object-oriented language **Java**
- **Understand** the underlying **concepts** and **models** of programming languages to be able to learn other languages independently
- **Apply problem solving concepts** based on data structures, objects, classes, methods, lists, queues, recursion

Learning goals (detailed)

- **Apply** basic notions to solve problems using algorithms and programs
- **Understand** imperative programming constructs
- **Differentiate** syntax and semantics
- **Understand** the syntax of programming languages based on regular expressions and context-free grammar
- **Understand** the semantics of programs using control-flow graphs
- **Understand** numbers, strings, arrays, insertion sort
- **Understand** and **apply** recursion (binary search, patterns of recursion)
- **Implement** classes, attributes, methods and **instantiate** objects
- **Use** data structures such as lists, stacks, queues
- Object-oriented programming: **apply** inheritance, abstract classes, interfaces, polymorphism
- **Understand** the perspectives for programming in the large (software engineering)

Roadmap

- **Context**
  - You have your own laptop and can use it during workshops and tutorial groups!
  - You are motivated to learn programming
- **Learning goals**
  - Understand the organization of workshops and exercises
  - Interact with us on Artemis
  - Participate in onsite tutorial group meetings
  - Understand a first small program
  - Create simple classes with attributes and methods
  - Instantiate objects and invoke methods
  - Understand the most important data types

Outline

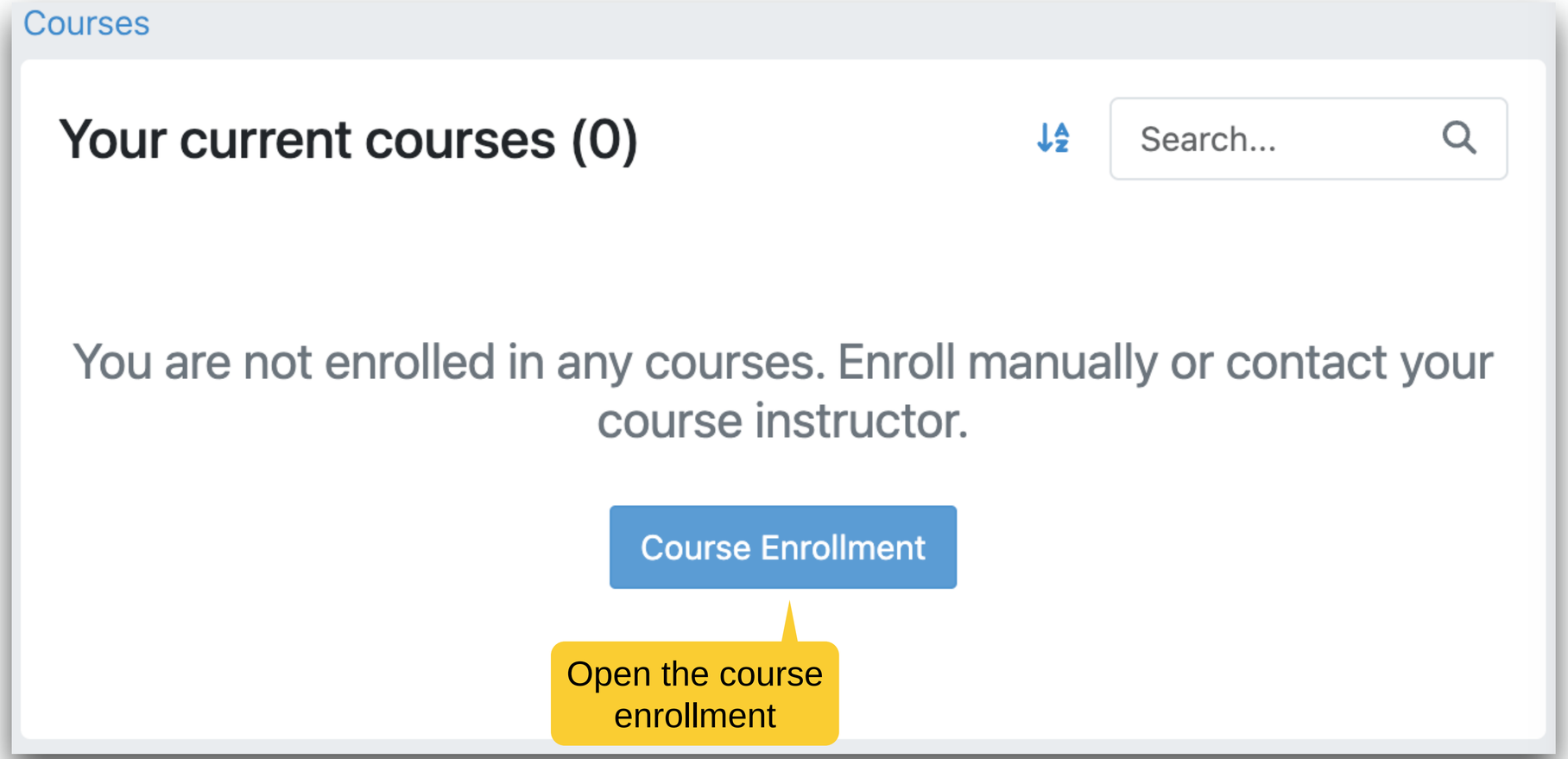
- **General organization of the course**
- Object oriented programming
- Basic types and their operations
- Methods
- UML class diagrams

Tools

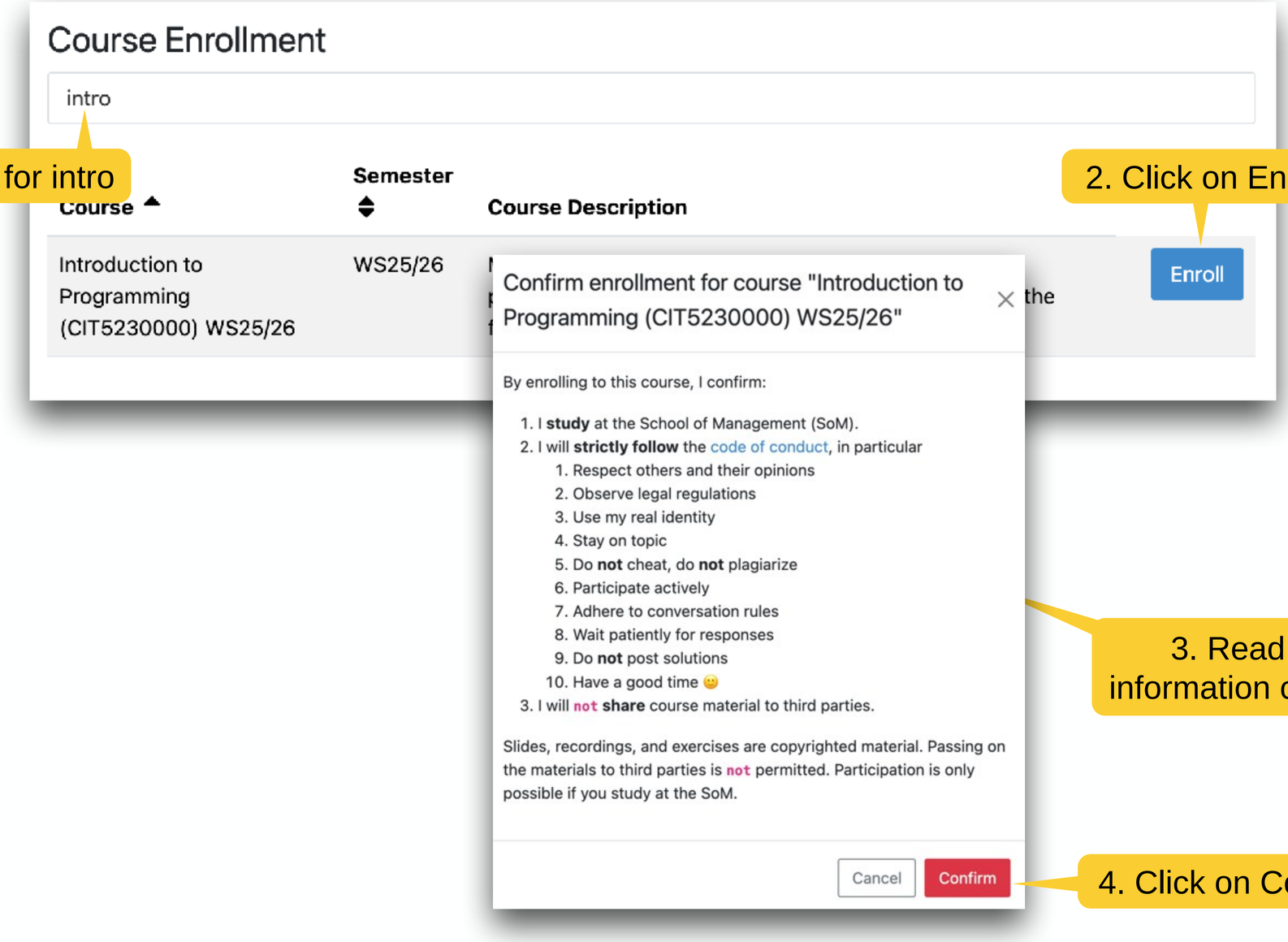
Artemis: https://artemis.tum.de

- Download slides before the workshop starts
- Participate in exercises: programming, modeling, quiz, text
- Review your individual exercise results including feedback
- Participate in exams (supervised exercises)
- Communicate within tutorial groups
- Discuss with other students
- Read announcements
- Ask questions about the general organization
- Find answers to typical questions (FAQs)
- Post questions and answers related to specific workshops and exercises

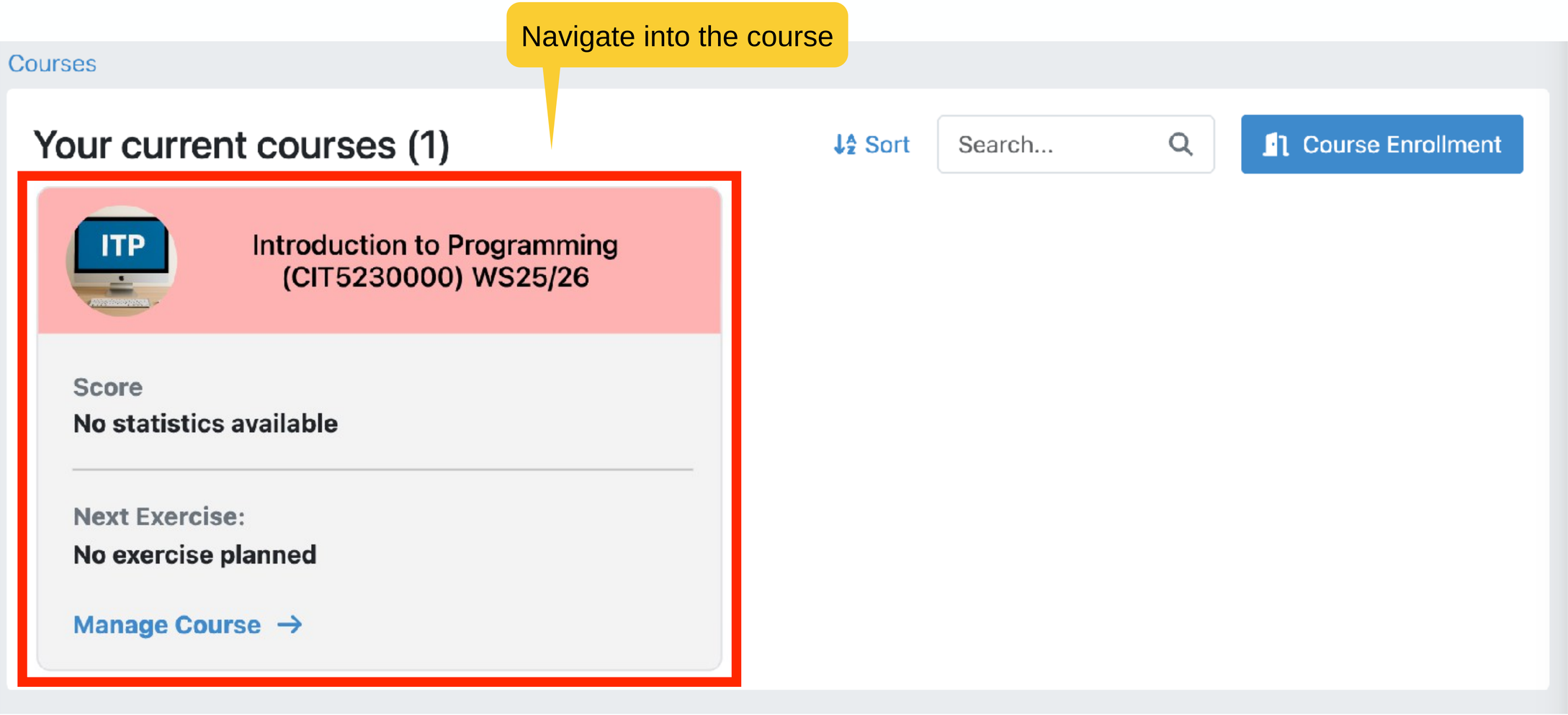
Task 1: Navigate into the course



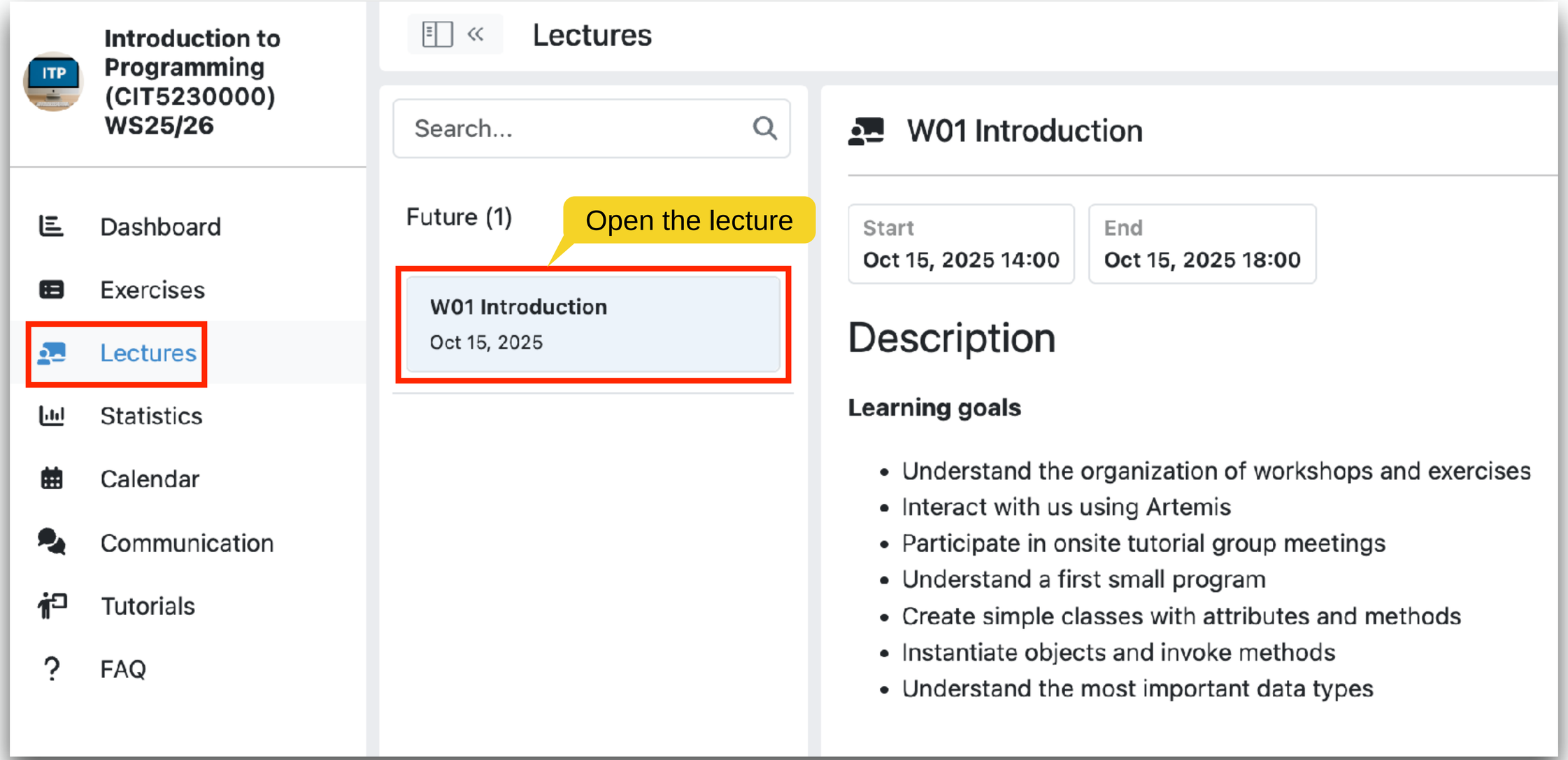
Task 1: Navigate into the course



Task 1: Navigate into the course



Task 2: Navigate into the first “lecture”



Task 3: Download the slides

Description

Learning goals

- Understand the organization of lectures and exercises
- Interact with us using Artemis and Discord
- Participate in on-site tutor meetings
- Understand a first small program
- Create simple classes with attributes and methods
- Instantiate objects
- Understand the most important data types

Lecture Units

Download Complete PDF

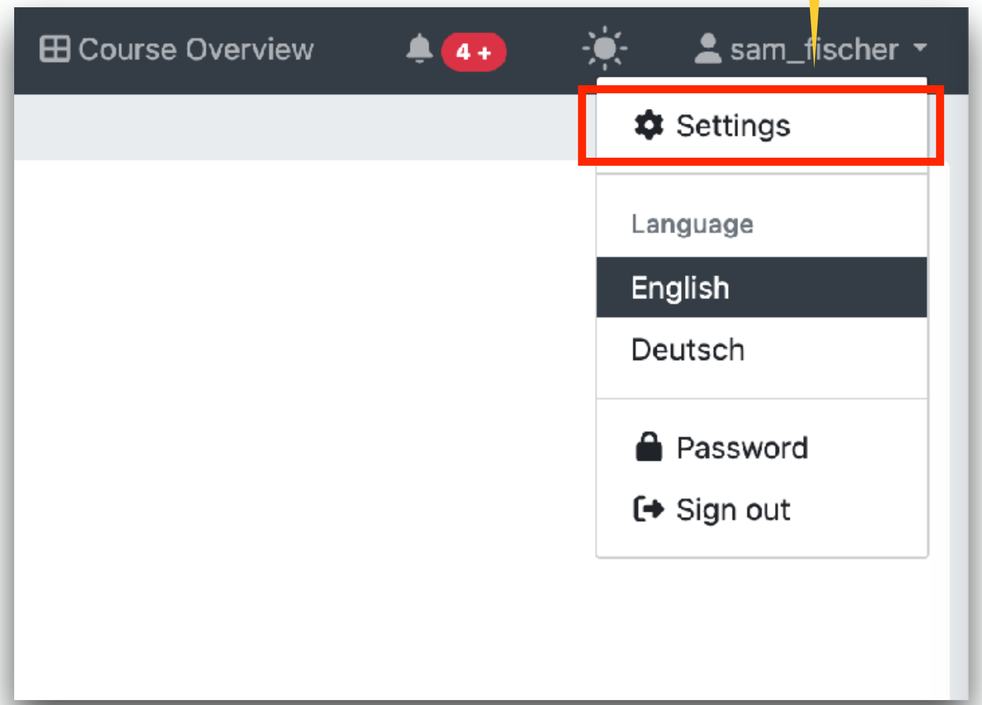
L01U01 General organization Download

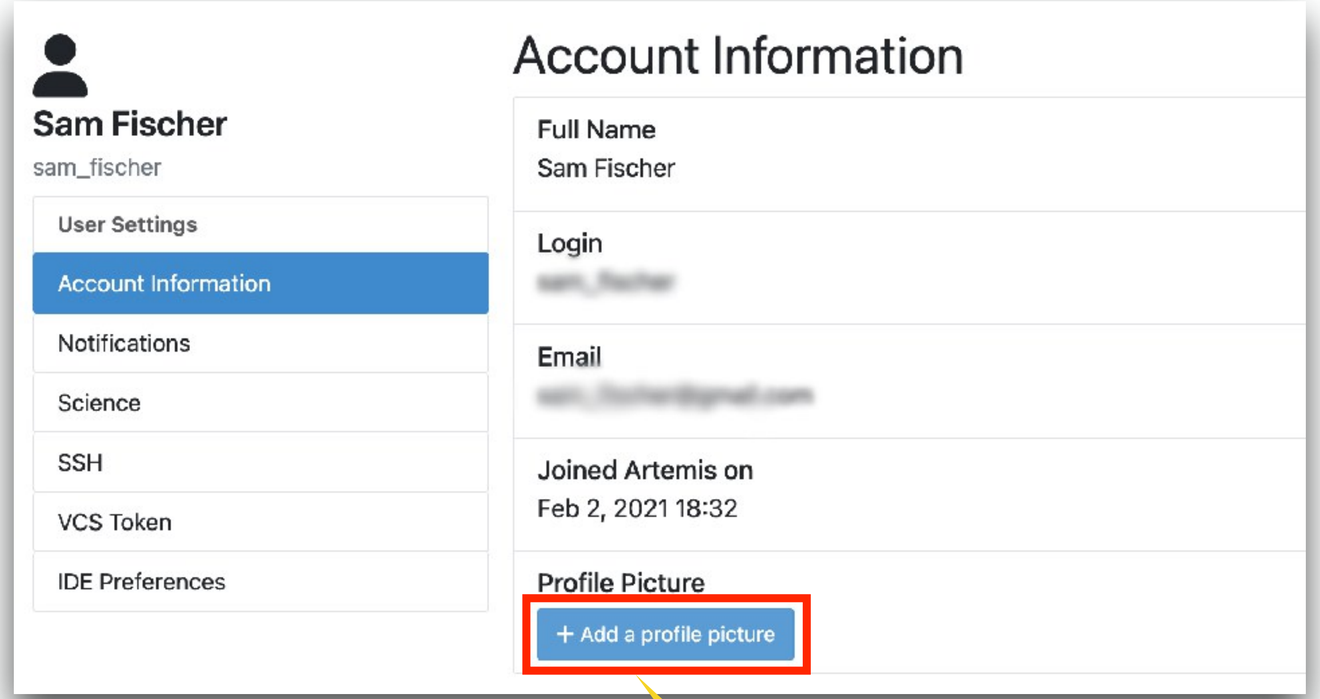
Upload Date: Oct 18, 2023 13:41
Version: 2
Filename: ITP2324_L01U01_General_organization.pdf

Download the slides

Task 4: Add your profile picture

1. Go to settings

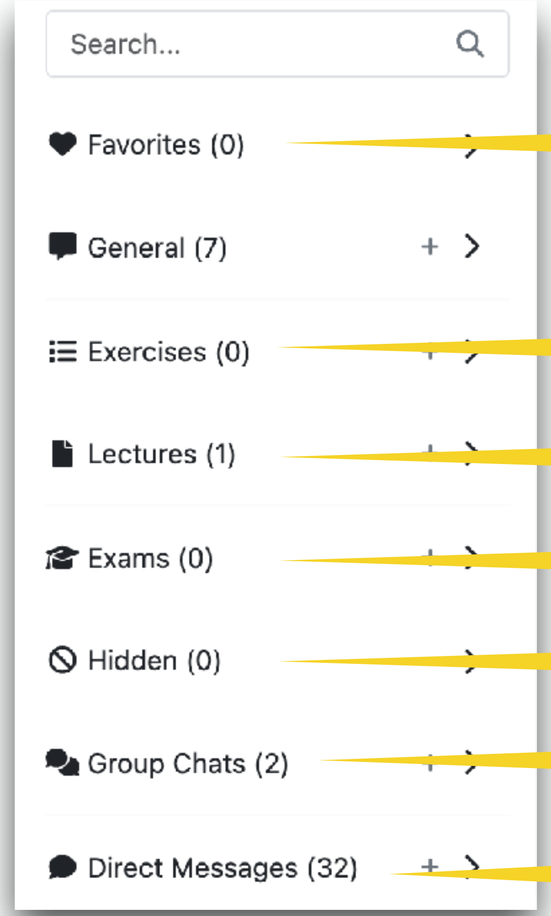




2. Add a profile picture

Course communication

- Overview of channels



Mark a channel as favorite by clicking the 🖤 next to it

Questions specific to **exercises**

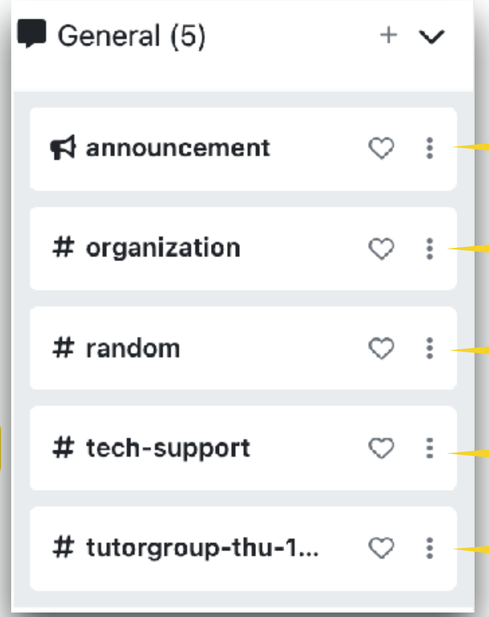
Questions specific to **workshops**

Questions specific to **exams**

Hidden channels

Your group chats

Private messages



Announcements from instructors

Questions regarding the organization

For your fun

Questions regarding tools and technical issues

Your tutorial group channel

→ Use these channels for their **intended purposes**

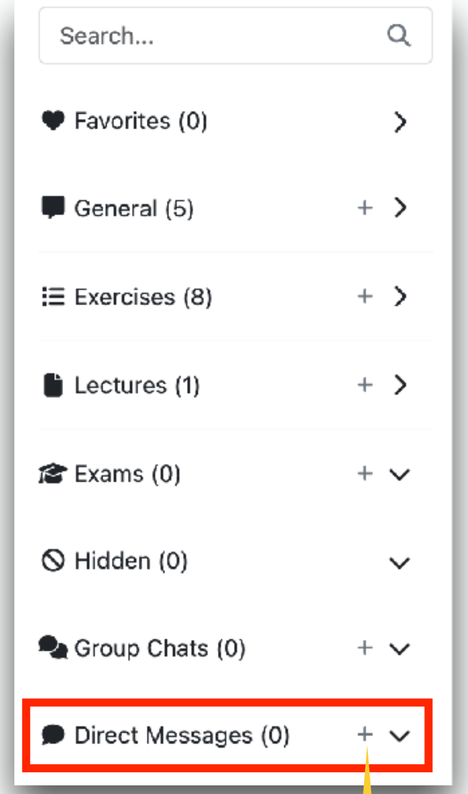
→ The purpose is to **enable communication**, ask questions, and share the answers with everyone

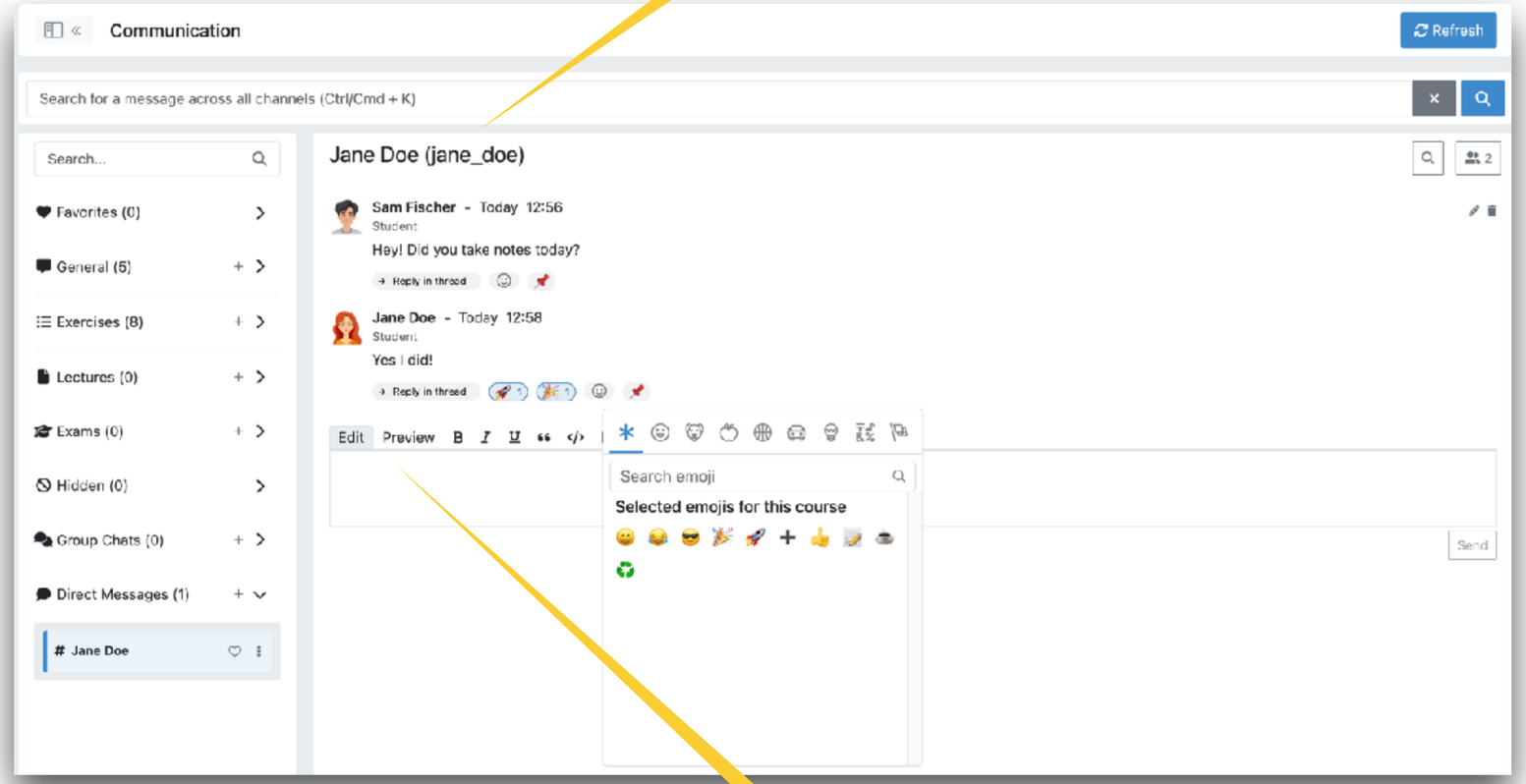
→ Do **not** offend and bully each other

Follow the code of conduct

Course communication

- Sending a private message





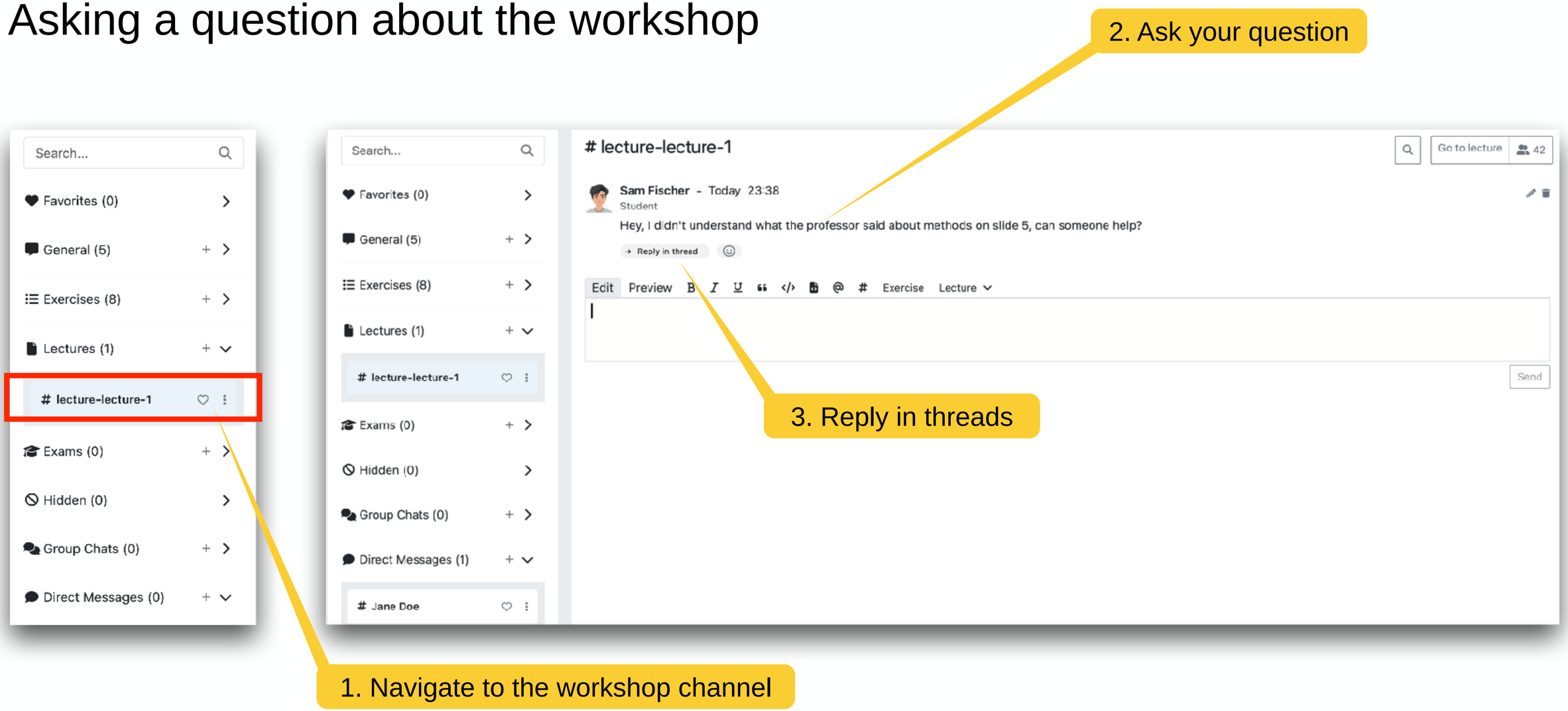
Your chat window

To send a private message, click on “+”

You can add reactions to messages, reply in separate threads, quote…

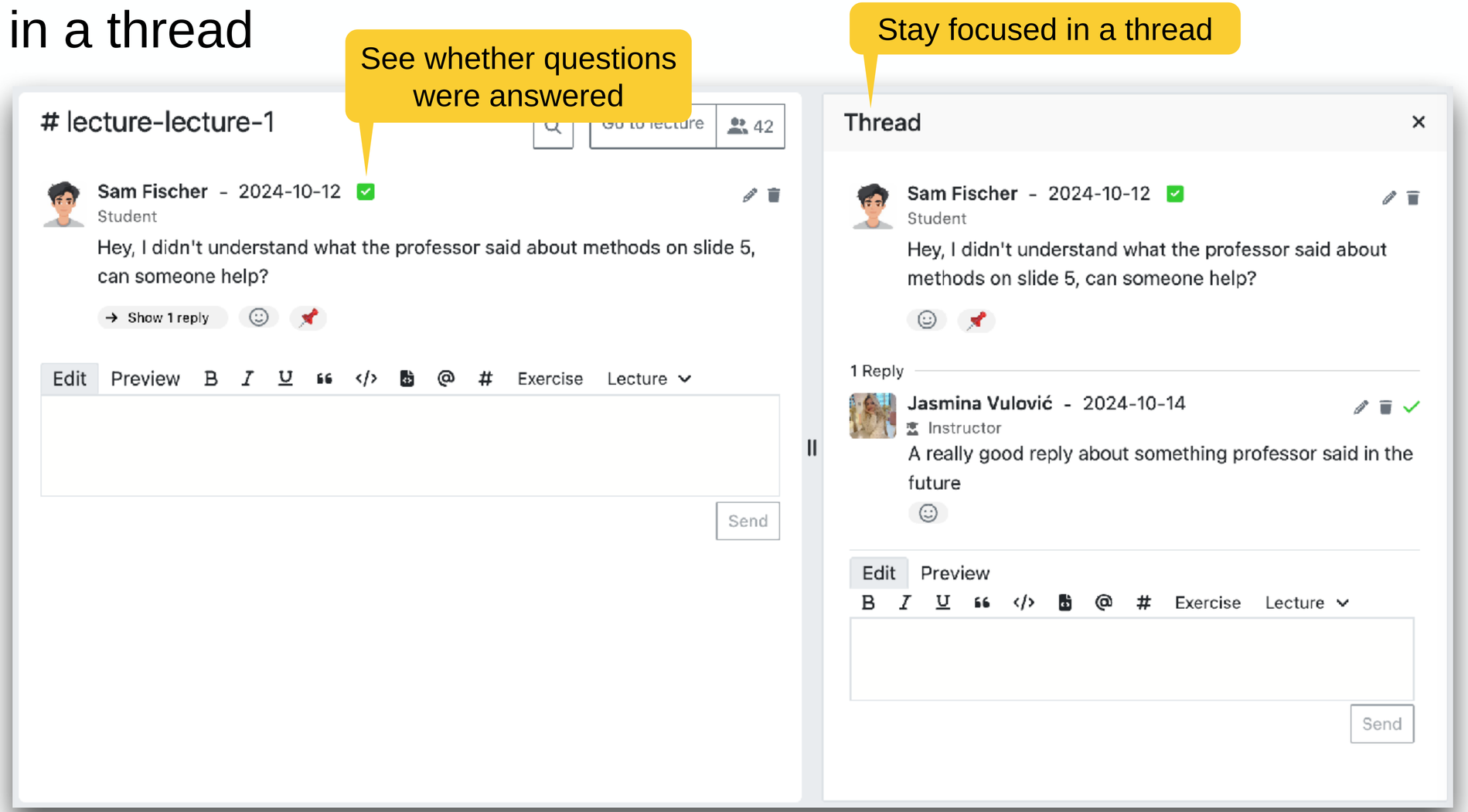
Course communication

- Asking a question about the workshop



Course communication

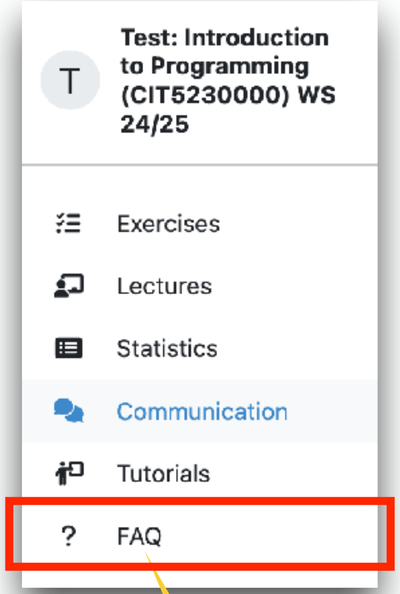
- Reply in a thread

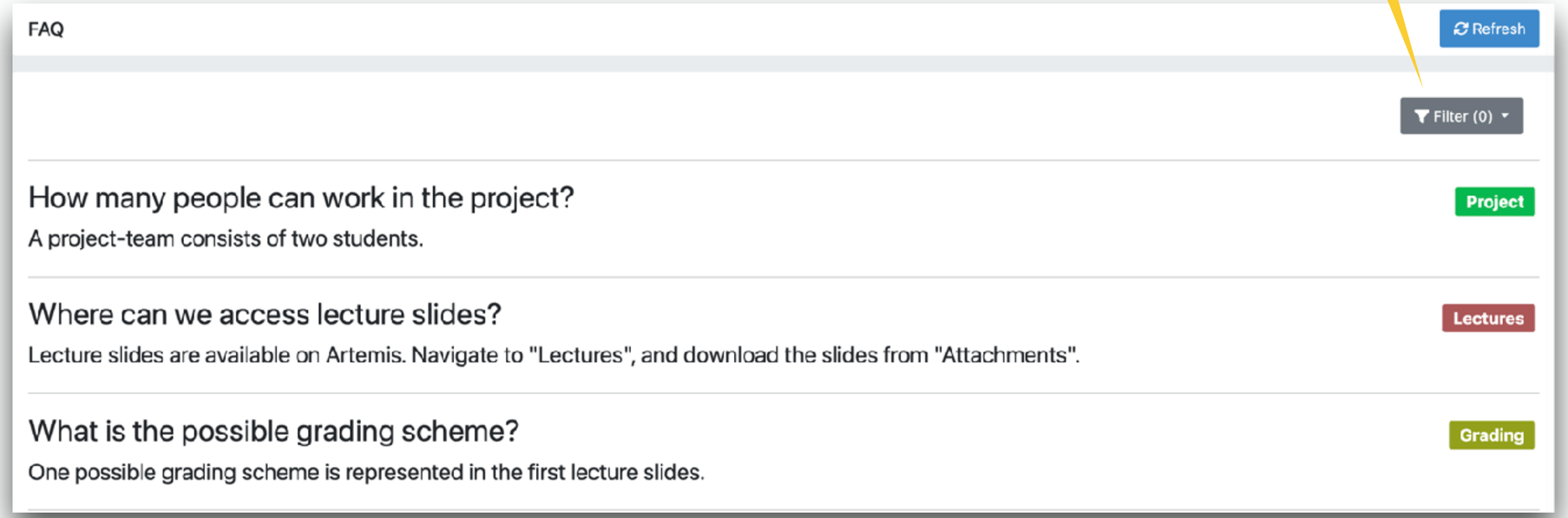


Course communication

- FAQs are a collection of frequently asked questions, where you can quickly find answers to common concerns

2. You can apply a filter based on your question





1. Navigate to the FAQ on the left sidebar

Artemis mobile apps

More Info

More Info

Download on the App Store

GET IT ON Google Play

Schedule

#	Date	Subject
1	**15.10.25**	**Introduction**
2	22.10.25	Control Flow in Programming
3	29.10.25	**In-Depth Core Concepts***
4	05.11.25	Core Data Structures
5	12.11.25	Code Reuse and Structure
6	19.11.25	Type Flexibility and Safety
7	26.11.25	**In-Depth Object Orientation***
8	03.12.25	Functional Programming Essentials
9	10.12.25	Algorithms and Data Handling
10	17.12.25	Programming Languages
11	07.01.26	Graphical User Interfaces
12	14.01.26	Recursion
13	21.01.26	Concurrency
14	28.01.26	Beyond Programming
15	04.02.26	Course Review

*** Repetition**

Teaching philosophy

“Tell me and I will forget.
Show me and I will remember.
Involve me and I will understand.
Step back and I will act.”

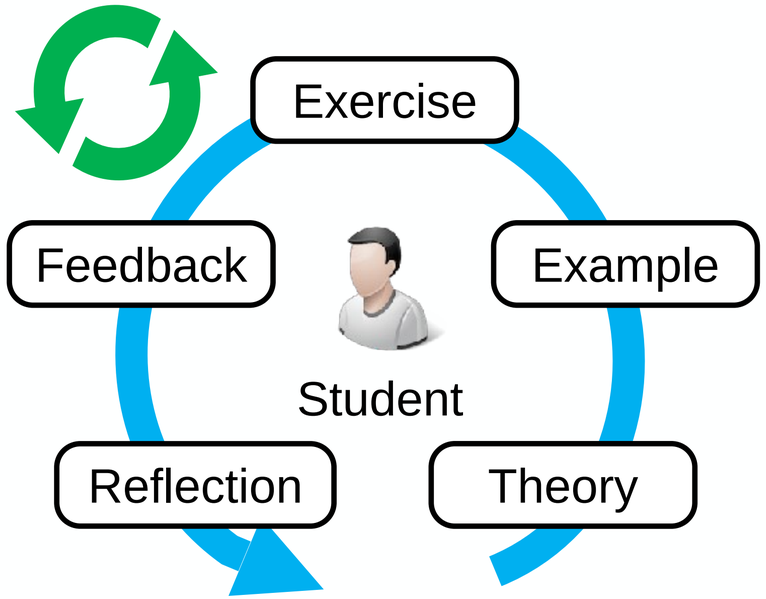
— Chinese Proverb

Interactive learning

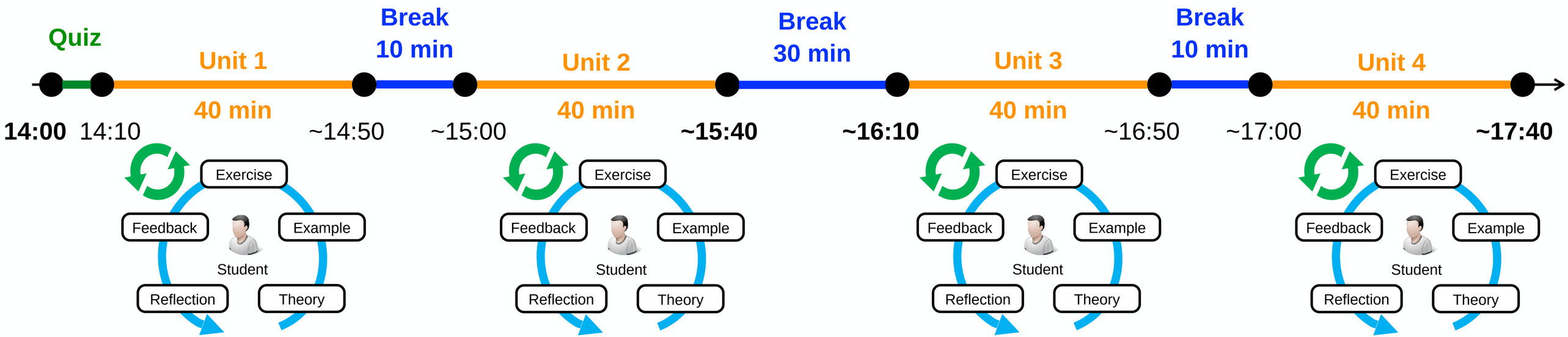
- **Goal:** directly involve students in the learning process using digital tools
  - Engage students in two aspects – **doing** things and **thinking** about those things
- Time slot: **Wednesday 14:00 - 18:00** with 4 x 40 min each week

  1) We teach you one (or more) concepts
  2) You apply these concepts in small **in-class exercises**
  3) You reflect on the concepts you are applying
- Artemis supports **interactive learning**
- Tutor groups deepen the knowledge
  - Group exercises
  - Homework presentation

Soft skills: **presentation** and **communication**



Workshop schedule



Copyright

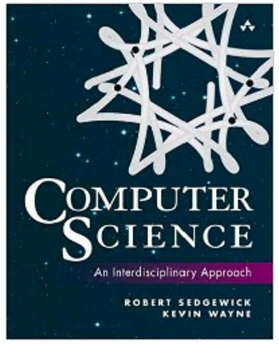
- All slides and exercise resources are copyright protected
- Slides are based on material by Helmut Seidl, Alexander Pretschner and Marco Kuhrmann
- You are **not** allowed to distribute them to other people
- You are **not** allowed to publish them on the internet

All material of this course is protected by copyright and has been copied by and solely for the educational purposes of the university under license. You may not sell, alter or further reproduce or distribute any part of this material to any other person. Where provided to you in electronic format, you may only print it for your own private study and research.

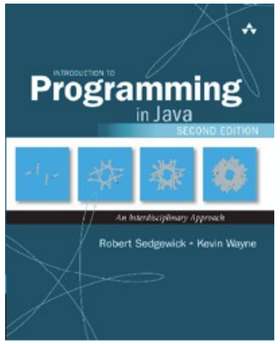
Failure to comply with the terms of this warning may expose you to legal action for copyright infringement and / or disciplinary action by the university.

Literature

- English books on computer science and Java programming



Sedgewick, Robert, and Kevin Wayne.
Computer science: An interdisciplinary approach.
Addison-Wesley Professional, 2016.



Sedgewick, Robert, and Kevin Wayne.
Introduction to programming in Java: an interdisciplinary approach.
Addison-Wesley Professional, 2017.

- Corresponding material can be found on https://introcs.cs.princeton.edu/java/home
- Specific references to literature are added in the slides if needed!

Overview of 35 tutorial group slots

First sessions for Thu-1/2/3-Mu-1 will exceptionally take place in Garching

Munich (city campus)

Time	Monday	Tuesday	Wednesday	Thursday	Friday
Morning Slot (09:00 - 12:15)	Mon-1-Mu-1 Mon-1-Mu-2 Mon-1-Mu-4	Tue-1-Mu-1 Tue-1-Mu-2		Thu-1-Mu-1 Thu-1-Mu-2	Fri-1-Mu-1 Fri-1-Mu-2 Fri-1-Mu-3
Afternoon Slot (12:30 - 15:45)	Mon-2-Mu-1 Mon-2-Mu-2	Tue-2-Mu-1 Tue-2-Mu-2		Thu-2-Mu-1 Thu-2-Mu-2	Fri-2-Mu-1 Fri-2-Mu-2 Fri-2-Mu-3
Evening Slot (16:00 - 19:15)	Mon-3-Mu-1 Mon-3-Mu-2 Mon-3-Mu-3	Tue-3-Mu-1 Tue-3-Mu-2		Thu-3-Mu-1 Thu-3-Mu-2	Fri-3-Mu-1 Fri-3-Mu-2

Garching

Time	Monday	Tuesday	Wednesday	Thursday	Friday
Morning Slot (09:00 - 12:15)	Mon-1-Ga-4	Tue-1-Ga-3			Fri-1-Ga-1
Afternoon Slot (12:30 - 15:45)				Thu-2-Ga-2	Fri-2-Ga-1
Evening Slot (16:00 - 19:15)		Tue-3-Ga-1		Thu-3-Ga-2	

Name Pattern:

Thu-1-Mu-2

- Thu → Weekday
- 1 → Slot Number
- Mu → Campus
- 2 → Room

ID	Time
1	09:00 - 12:25
2	12:30 - 15:45
3	16:00 - 19:15

ID	Room
Mu-1	0510
Mu-2	1237
Mu-3	Z510
Mu-4	Z538
Ga-1	01.07.014
Ga-2	01.13.010
Ga-3	00.08.053
Ga-4	00.08.055

Registration for tutorial groups

- URL to register and enter your priorities: https://campus.tum.de/tumonline/ee/ui/ca2/app/desktop/#/slc.tm.cp/student/courses/950841407
- Timeline
  - Enter your tutorial group preferences **before today, 15:30 (October 15)**
  - Receive your assignment today on TUMonline
  - The first meetings take place **tomorrow, Thursday, October 16**
- Exception: if you missed the matching, could not be matched or want to change your group
  - Late registration or a change of the tutorial group will be possible until **Friday (October 24)** on TUMonline (there is a daily matching to assign new students)
  - Contact your fellow students on **Artemis** if you want to switch
  - **Please note:** changing a group is only possible with a good reason

Tutor group rules

- Make sure to attend the tutorial groups **onsite**!
- In case attending the tutorial group is not possible, e.g.
  - Meeting is cancelled (SVV, FVV, holiday, etc.)
  - You are ill
  - Your tutor is ill
  - The tutorial group is canceled completely due to small participation

→**Reallocate yourself to another tutorial group**

Code of conduct

1 Honesty
We value honesty as fundamental to our community, promoting open communication and sincerity in our interactions, building a culture of trust.

2 Reliability
We create reliability by complying with our obligations and being trustworthy, ensuring others can confidently depend on us in all circumstances.

3 Trust
Trust grows from openness, mutual respect and honesty, creating a supportive and safe environment where collaboration and mutual growth can flourish.

4 Fairness
We commit to impartial decisions, free from favoritism or self-interest, treating everyone fairly and respectfully, recognizing diverse perspectives.

5 Diversity
We embrace diversity, respecting all kinds of backgrounds, cultures and beliefs, valuing the unique perspectives and insights they bring to our community.

6 Equity
We uphold equity by addressing individual needs, actively removing barriers. We make sure that everyone has an equal opportunity to succeed in our community.

7 Sustainability
Sustainability guides our actions as we make mindful, responsible choices that minimize environmental impact, protect the planet, and support a healthy future.

8 Responsibility
Responsibility means owning our actions and their impact on others, understanding our mission toward building a better, ethical, and more inclusive society.

9 Respect
We honor the dignity of every individual by encouraging understanding, kindness, and empathy in all interactions in order to foster respectful interaction.

10 Evidence
We are fully committed to a science-based approach that promotes informed decisions, challenges established assumptions and supports continuous learning.

Violations may result in a permanent exclusion from attending tutorial groups, interacting on Artemis, or even participating in the course!

You can find more details on https://aet.cit.tum.de/code-of-conduct

Code of conduct (specifically for this course)

1. Respect others and their opinions
2. Observe legal regulations
3. Use your real identity
4. Stay on topic
5. Do **not** cheat, do **not** plagiarize
6. Participate actively
7. Adhere to conversation rules
8. Wait patiently for responses
9. Do **not** post solutions
10. Have a good time 🙂

Violations may result in a permanent exclusion from attending tutorial groups, interacting on Artemis, or even participating in the course!

You can find more details on https://aet.cit.tum.de/code-of-conduct

Typical schedule of the tutorial group meetings

1. **Short review (max. 5 min)** of the workshop content (**no repetition**)
   - Your tutor quickly shows the roadmap, outline and summary slides of the workshop
   - You can ask questions, if something is unclear (prepare them in advance)
2. **Presentation and discussion of previous homework (20 - 30 min)**
   - Present your solution to your tutor and the other students
3. **Group exercises (120 - 150 min)**
   - Solve exercises in a group and present your solution
4. **Short discussion of the new homework (max. 5 min)**
   - Your tutor quickly shows the new homework
   - You can ask questions, if something is unclear (prepare them in advance)

Grading

Preliminary dates

Activity	Weight
Supervised Exercise 1 (Tuesday, 02.12.2025, 19:00 - 20:40)	30 %
Supervised Exercise 2 (Monday, 09.02.2026, 19:00 - 20:40)	30 %
Team project: development of a game (January)	30 %
Presentations (homework + tutor exercise)	10 %
Total	**100 %**
Bonus for selected workshop and homework exercises	up to 5%

Computer-based exam in the lecture hall

Everything on Artemis with a **bonus** category

More information will be announced later

Possible grading key*

- Sum of weighted scores (based on previous slide)

Score in %	➡	Grade
[0.0, 40.0)	➡	5.0
[40.0, 45.0)	➡	4.7
[45.0, 50.0)	➡	4.3
[50.0, 55.0)	➡	4.0
[55.0, 60.0)	➡	3.7
[60.0, 65.0)	➡	3.3
[65.0, 70.0)	➡	3.0
[70.0, 75.0)	➡	2.7
[75.0, 80.0)	➡	2.3
[80.0, 85.0)	➡	2.0
[85.0, 90.0)	➡	1.7
[90.0, 95.0)	➡	1.3
[95.0, 100.0]	➡	1.0

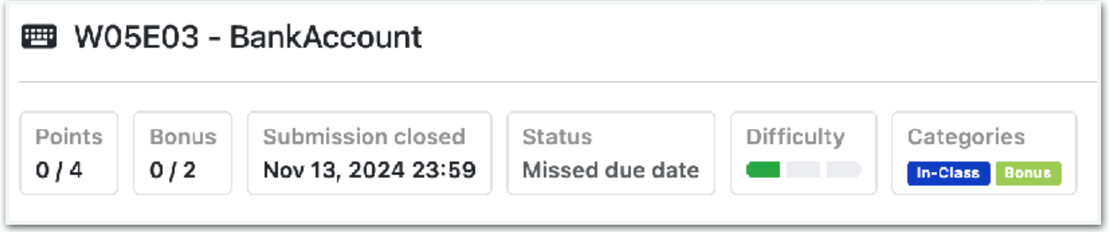
$(a, b] = \{ x \mid a < x \leq b \}$ $[a, b) = \{ x \mid a \leq x < b \}$

Rounding: to the nearest first decimal place (half up)

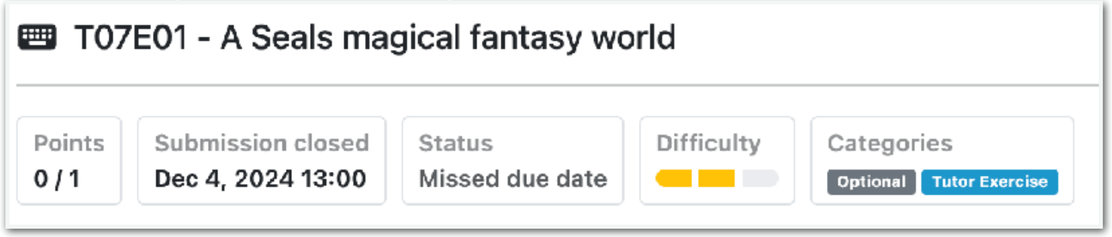
* subject to change

Bonus system

- Participate in workshop and homework exercises to receive points 🏆
- Exercises are marked with a **Bonus** category



- Other (non-bonus) exercises are marked as **Optional**



- Mapping of **exercise score** to **grade score**
  - Prerequisite: you have passed the course

Exercise score	➡	Additional grade score
[0%, 10%)	➡	0.0%
[10%, 20%)	➡	0.5%
[20%, 30%)	➡	1.0%
...	...	...
[80%, 90%)	➡	4.0%
[90%, 100%)	➡	4.5%
[100%,...%	➡	5.0%

$(a, b] = \{ x \mid a < x \leq b \}$

$[a, b) = \{ x \mid a \leq x < b \}$

→**Important:** the bonus is **not** applicable to the retake

Exercise types

1. Workshop exercises (e.g. W01E01)

2. Tutorial group exercises (e.g. T01E01)

- Solve exercises in a group during the tutorial group meeting
- Presented in the same tutorial groups
- Prepare for homework exercises and for the final exam

3. Homework exercises (e.g. H01E01)

- Published every **Wednesday evening**: to be solved within one week at home
- Assessed automatically (real time feedback)
- Presented and discussed in the subsequent tutorial group

Exercise rules

- You can only submit solutions to any type of exercise on Artemis (including team project)
- Each exercise has a due date
  - If you miss the deadline, your solution will **not** be graded
  - You can practice the exercise after the due date (without consideration for bonus)
- **Important:** exercises must be completed and submitted **independently**
  - You need a unique solution for each exercise
  - Plagiarism, i.e. copying blindly without understanding the concepts and without applying the skills, might be tempting, but you basically betray yourself

🤖 AI policy

- AI Tools are no requirement to pass the course! **Use them responsibly!**

- 🔴 **Not allowed**: AI tools are **not** permitted during supervised exercises
- 🟡 **Use responsibly**: you may use GitHub Copilot* for programming tasks during exercises, homework, and the team project
  - **❗ *It must be disabled during supervised exercises; otherwise, it will be considered a cheating attempt**
- 🟢 **Recommended**: we encourage you to use **Iris** for all programming tasks during exercises, homework, and the team project on Artemis

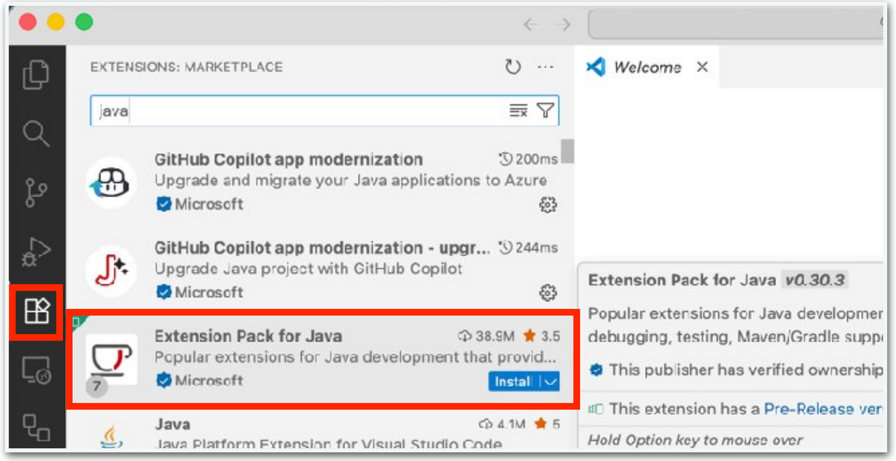
More about Iris in your first tutorial group

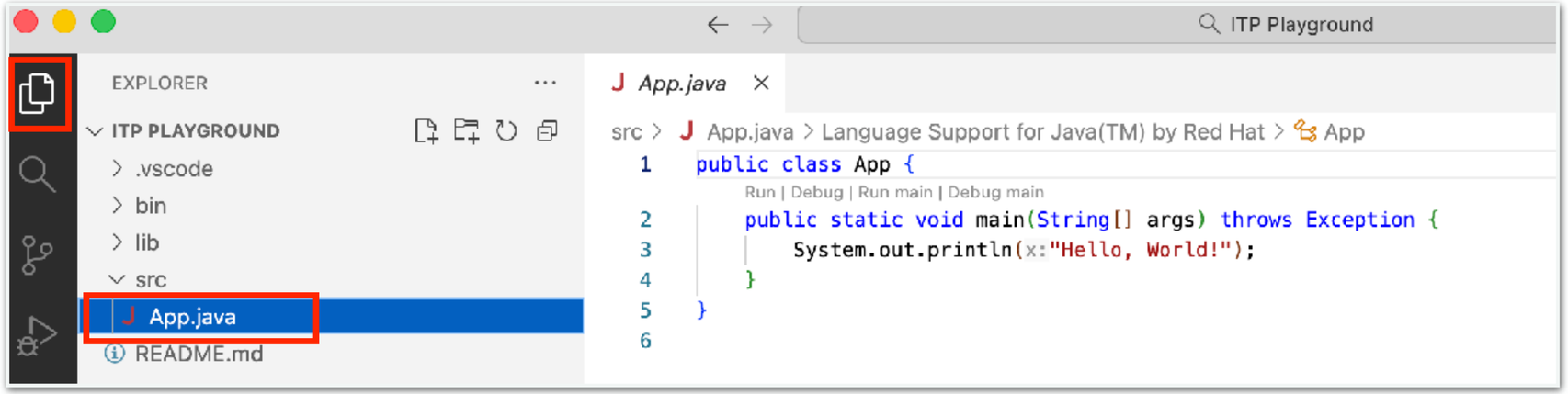
Outline

- General organization of the course
- **Object oriented programming**
- Basic types and their operations
- Methods
- UML class diagrams

VS Code playground

- Let’s open Visual Studio Code
- Install “**Extension Pack for Java**” from the Marketplace
- Create a new Java project called **ITP Playground**
- Use the Command Palette (Shift + Cmd + P)
- Choose “**Java: Create Java Project…**” → “**No build tools**”
- Select a folder called **ITP Playground** in your file system
- Open the file **`App.java`**





→ Please code the examples from the slides into the Playground project

Object oriented programming

- The world consists of objects
- Basic idea
  - Every object in reality has a virtual counterpart
  - Objects cooperate through data exchange and invocation of services
- Perspective
  - "Who does what?" → Properties and capabilities of objects
  - Algorithmic perspective: "How is it done?"

Example: library

Class (type)

Book
String author String title int edition ...
void borrow(Date, Date) void giveBack(Date) ...

Attributes

Methods

Coding

Java class

```
class Book {
    String author;
    String title;
    int edition;
    //...
    void borrow(Date s, Date e) {
        //...
    }
    void giveBack(Date today) {
        //...
    }
    //...
}
```

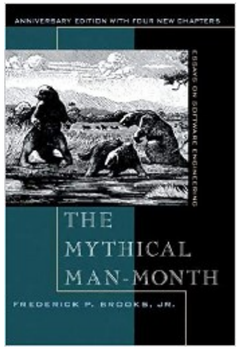
Code comment
// + green

Usage

```
Book mmm = new Book();
mmm.author = "Fred Brooks";
mmm.title = "The mythical man ...";
mmm.edition = 2;
```

Creation

Generalization



Object of the real world

Modeling

mmm:Book
author="Fred Brooks" title="The mythical man month" edition=2 ...

Object (instance)

Object model

Definition: object and class

- Three aspects characterize objects
  - **Identity:** properties can change, identity is constant
    Ensured by **name** (reference)
  - **State:** set of properties at a point in time
    Realized by **attributes**
  - **Behavior:** execution of actions that change the state
    Realized by **methods**
- A class is a "blueprint" for similar objects and determines
  - Which attributes the objects have
  - Which methods the objects have

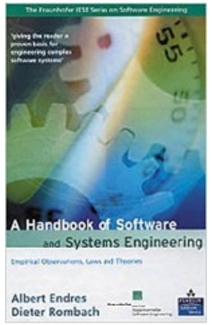
Classes in Java

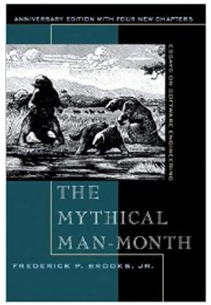
```java
import java.util.Date;

class Book {
    // Attributes
    String author;
    String title;
    int edition;
    //...

    // Constructor
    Book() {
        //... initialize the object
    }

    // Methods
    void borrow(Date s, Date e) {
        //...
    }
    void giveBack(Date today) {
        //...
    }
    //...
}
```

Schema

```
class Book {
  body
}
```

Example: library





hse:Book

author=“Dieter Rombach …”
title=“A handbook of …”
edition=1

csr:Book

author=“Stacy Powell et al.”
title=“Cleanroom software …”
edition=1

mmm:Book

author="Fred Brooks"
title="The mythical man month"
edition=2

Object model

Book

String author
String title
int edition
…

void borrow(Date, Date)
void giveBack(Date)
…

There can be several objects (instances) of a class (type) !

Example: library

Object of the real world

hse:Book

author="Dieter Rombach …"
title="A handbook of …"
edition=1

csr:Book

author="Stacy Powell et al."
title="Cleanroom software …"
edition=1

mmm:Book

author="Fred Brooks"
title="The mythical man month"
edition=2

hnlib:Library

location="Campus …"
openingHours=[9:00, 18:00]
books=[hse, csr, mmm];

Object (instance)

Different objects belong together

Association

Multiplicity

*

Library

~~Book[] books~~
String location
Time[] openingHours
…

void visit()
…

Book

String author
String title
int edition
...

void borrow(date, date)
void giveBack(date)
…

Class (type)

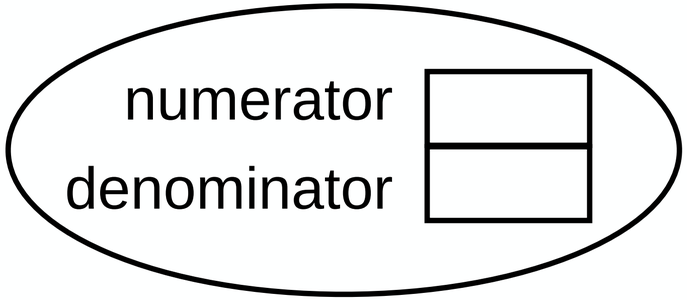
Variables

- The attributes (data components) of an object are also called instance variables
- Later we will learn about other variables that are defined for
  - Classes
  - Methods (locally defined)
- Variables must be declared
- A constructor must be called (except for so-called basic types)

Example: rational numbers

- A rational number q ∈ Q has the form q = $\frac{x}{y}$, where $x, y \in Z$
- x and y are called numerator and denominator of q
- An object of type **`Rational`** should contain two `int` variables **`numerator`** and **`denominator`** as components (attributes)

Object



Example: rational numbers

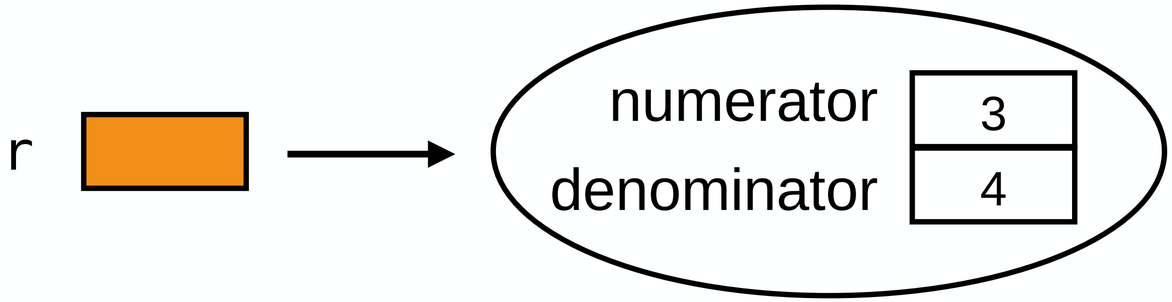
Code

```
class Rational {
    int numerator;        // Declaration of an int attribute
    int denominator;      // Declaration of an int attribute

    // constructor Rational() exists automatically

    // methods . . .
}
```

Declaration and constructor

- **`Rational name;`** declares a variable for objects of the class `Rational`
- The command `new Rational(...)` creates the object, calls a constructor for this object and returns the new object

```
Rational r = new Rational();
r.numerator = 3;
r.denominator = 4;
```



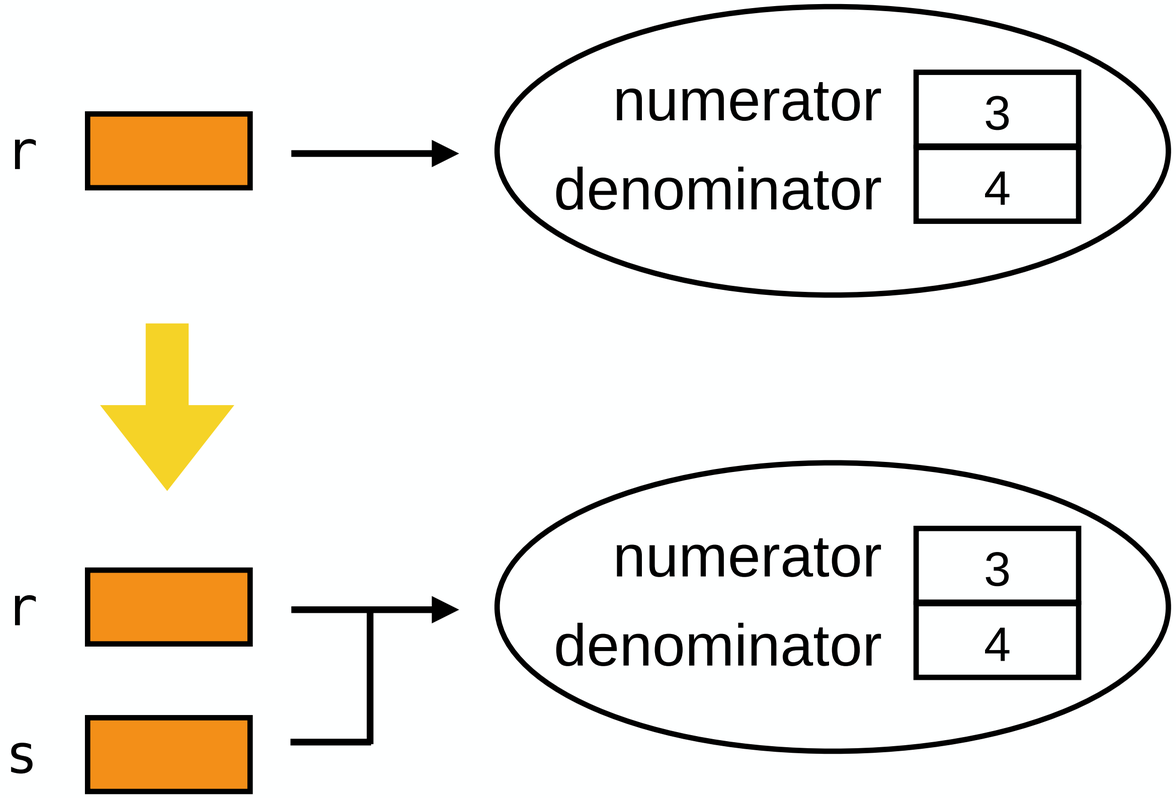
Constructors

- Special method that reserves memory in the context of **`new`** (and can initialize attributes of the new object)
- Always carry the name of their class
- A default constructor **`ClassName()`** in the class **`ClassName`** is provided automatically (if no other constructor is defined)
- Can also have arguments
- If you add a custom constructor (with parameters), the default one is not automatically available anymore

Object references

- The value of a **`Rational`** variable is a reference
- `Rational s = r;` copies the reference from `r` to the variable `s`

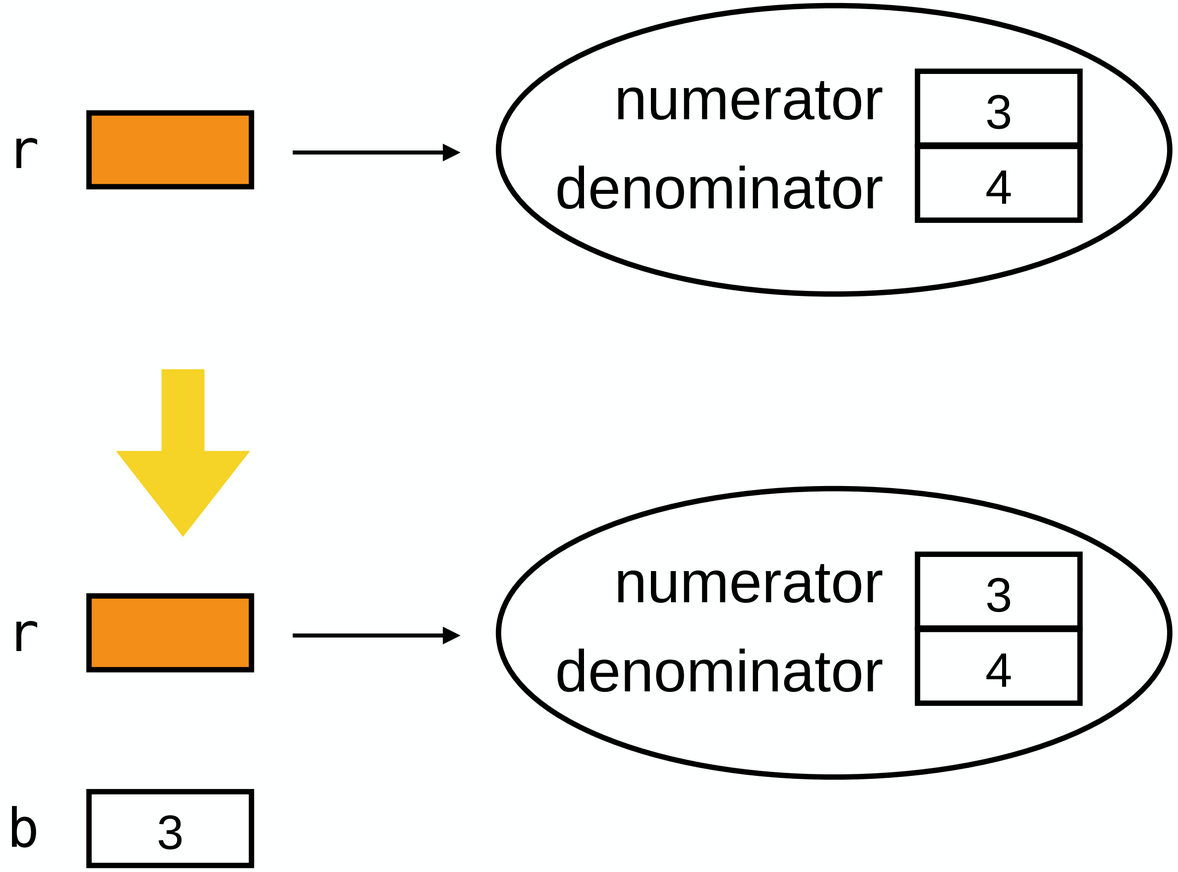
```
Rational s = r;
```



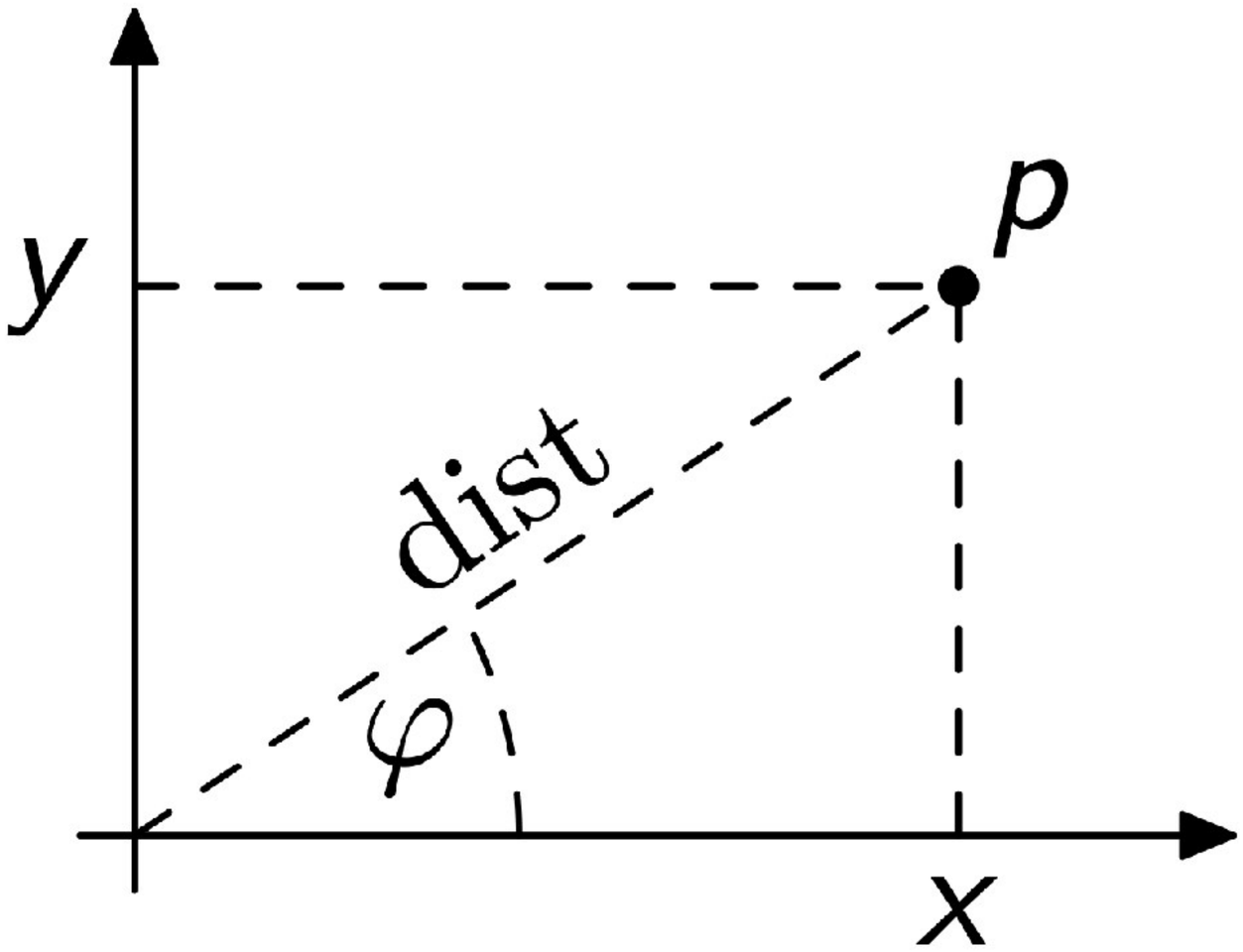
Access to attributes

- `r.numerator` returns the value of the `numerator` attribute of the object `r`

```
int b = r.numerator;
```



Example: points in R^2



Example: points in R^2

Code

```
class Point {
    double x;
    double y;

    // constructor Point() exists automatically

    // methods . . .
}
```

Create points / references

- Create points with the keywords **`new`**
  - `Point p = new Point(); // create Point p`
  - `Point q = new Point(); // create Point q`
  - `p, q` are names for references to new objects of the class **`Point`**
- A point can have multiple identifiers
  - `Point r = p; // r, p reference the same object`

Functionality of references (1)

- Consider two declarations of local variables
  - `Point p;`
  - `int n;`
- Both variables are not (yet) initialized

Question: what is their current (default) value?

- Initialize
  - `p = new Point();`
  - `n = 5;`
- `new Point()` creates an object of type `Point` (on the heap) and returns a reference to the object
- This reference is stored in the reference variable `p`
- `n = 5` stores the value `5` directly in the variable `n`

Functionality of references (2)

- Consider the following variables

```
Point p = new Point();
Point q = p;
```

Store the **value** of p in the variable q

```
int n = 5;
int k = n;
```

Store the **value** of n in the variable k

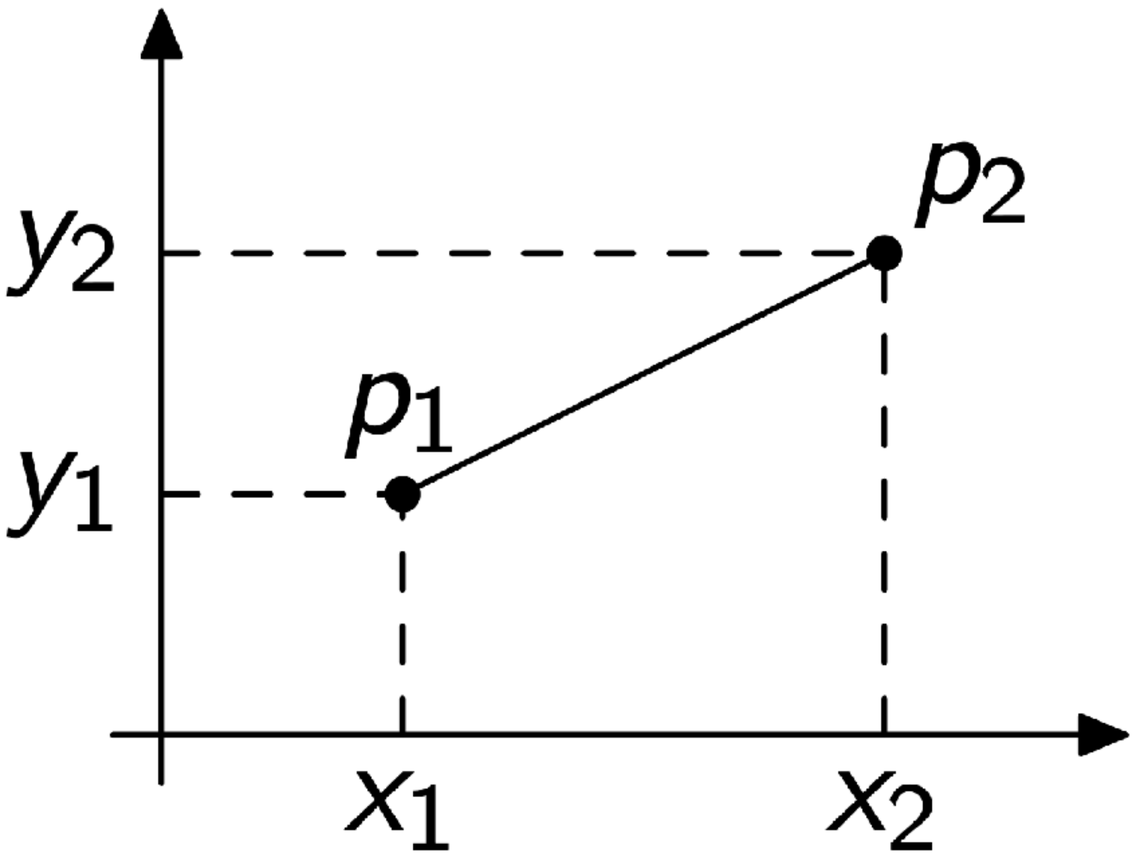
- In case of a p and q, the variables are references to the respective **object**

  → p and q refer to one and the same object

- In case of a n and k (primitive, basic types), the variables hold the value itself

  → n and k are independent variables (coincidentally they have the same value 5)

Example: objects as attributes of objects: lines in R^2



Example: objects as attributes of objects: lines in R^2

Code

```
class Line {
    // attributes: end points
    Point p1;
    Point p2;

    // Default constructor Line() no longer exists
    // if another constructor is defined.
    // Here: constructor explicitly with two arguments!
    Line(Point p, Point q) {
        p1 = p;
        p2 = q;
    }

    // other methods . . .
}
```

Example: objects as attributes of objects: lines in R^2

Code: creation of objects

```
Point p1 = new Point();
p1.x = 2.0;
p1.y = 2.7;

Point p2 = new Point();
p2.x = 5.8;
p2.y = 16.0;

Line l = new Line(p1, p2);
```

Improved example with a constructor

```
Point p1 = new Point(2.0, 2.7);
Point p2 = new Point(5.8, 16.0);
Line l = new Line(p1, p2);
```

```
class Point {
    double x;
    double y;

    Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // methods . . .
}
```

Refers to the current object

`null`

- The default value for all reference types is **`null`**
- If a reference variable or a reference attribute have not been initialized, then it has the value **`null`**
- If a variable has the value **`null`**, it does not refer to any object
  - Example: `Point p; // now applies: p == null`
  - Access to **`p.x`** and **`p.y`** with **`p == null`** leads to runtime error!

NullPointerException

Information hiding

- Important concept in computer science
  - Why are endpoints not represented by two coordinate pairs `x1, y1` and `x2, y2` in class **`Line`**?
  - Change of implementation of **`Point`** would require a change of implementation of **`Line`** (e.g. when changing to polar coordinates)

→ The implementation of **`Points`** is the internal secret of the class **`Point`** alone

- This is the only way to limit the effects of changes locally (**principle of locality**)

Intermediate summary

- The (virtual) world consists of objects
- Objects are instances of classes, classes are blueprints / templates for objects
- Objects encapsulate data (attributes) and behavior (methods)
- Objects are accessed via references
  - References are defined by attribute declarations
  - Attributes can be objects themselves
- Objects are created / initialized by **`new`**
  - **`new`** always calls a constructor that performs initializations
- Two references can point to the same memory area (aliasing)

Outline

- General organization of the course
- Object oriented programming
- **Basic types and their operations**
- Methods
- UML class diagrams

Basic types

- A data type (short: type) defines a set of possible values and the possible operations (methods) on these types
- All variables and attributes have a type (and methods have a return type and types for the arguments)
- In Java, classes define data types (“classes are data types”)
- Java provides eight basic types (e.g. **`int`** and **`double`**)

Integer based types

Type	Space	Smallest value	Highest value
`byte`	8 bits	−128	127
`short`	16 bits	−32 768	32 767
`int`	32 bits	−2 147 483 648	2 147 483 647
`long`	64 bits	−9 223 372 036 854 775 808	9 223 372 036 854 775 807

Using smaller types like byte or short saves space

Note: Java does **not** warn in case of overflow or underflow!

Overflow example

```
int x = 2147483647; // largest int value
x = x + 1;          // overflow
System.out.println(x);
```

prints: -2147483648

Smallest **int** value

- You can assign a first value to a variable when you declare it directly (initialization)
- You can even declare it at the point where you need it for the first time

Floating point numbers

- There are two kinds of floating point numbers

Type	Space	Smallest value	Highest value	Precision
`float`	`32 bits`	`-3.4e+38`	`3.4e+38`	6 to 7 decimal digits
`double`	`64 bits`	`-1.7e+308`	`1.7e+308`	15 to 16 decimal digits

- Both double and float are approximate types, they are **not** precise
- Overflow/Underflow returns the values **`Infinity`** or **`-Infinity`**
- Consider the accuracy of the result to select the right type
- Floating point constants in the program are understood as `double`
- To differentiate, append the letter **`f`** (or **`F`**) or **`d`** (or **`D`**)

```
Example: 3.14f
```

Other basic types

Type	Space	Values
`boolean`	`1 bit`	`true, false`
`char`	`16 bits`	`all unicode characters`

- **Unicode** is a character set that includes all alphabets commonly used anywhere in the world, for example
  - The characters of our keyboard (including umlauts)
  - Chinese characters
  - Egyptian hieroglyphics
  - ...
- Char constants are written with single quotation marks: `'A', ';', '\n'`

Basic types and constructors

- Because they are used often and their implementation is efficient, no constructor is needed
- It is sufficient, for example, to write `int x = 2;`
- Only one memory area is reserved (for the value), not two as in the case of reference types (reference + object)

Equality and identity

- When are two objects of a basic type "equal"?
  - For basic types the value is stored directly, without reference
- Check the equality of the value for basic types therefore with **==**
- For reference types the reference is compared with **==**

Example

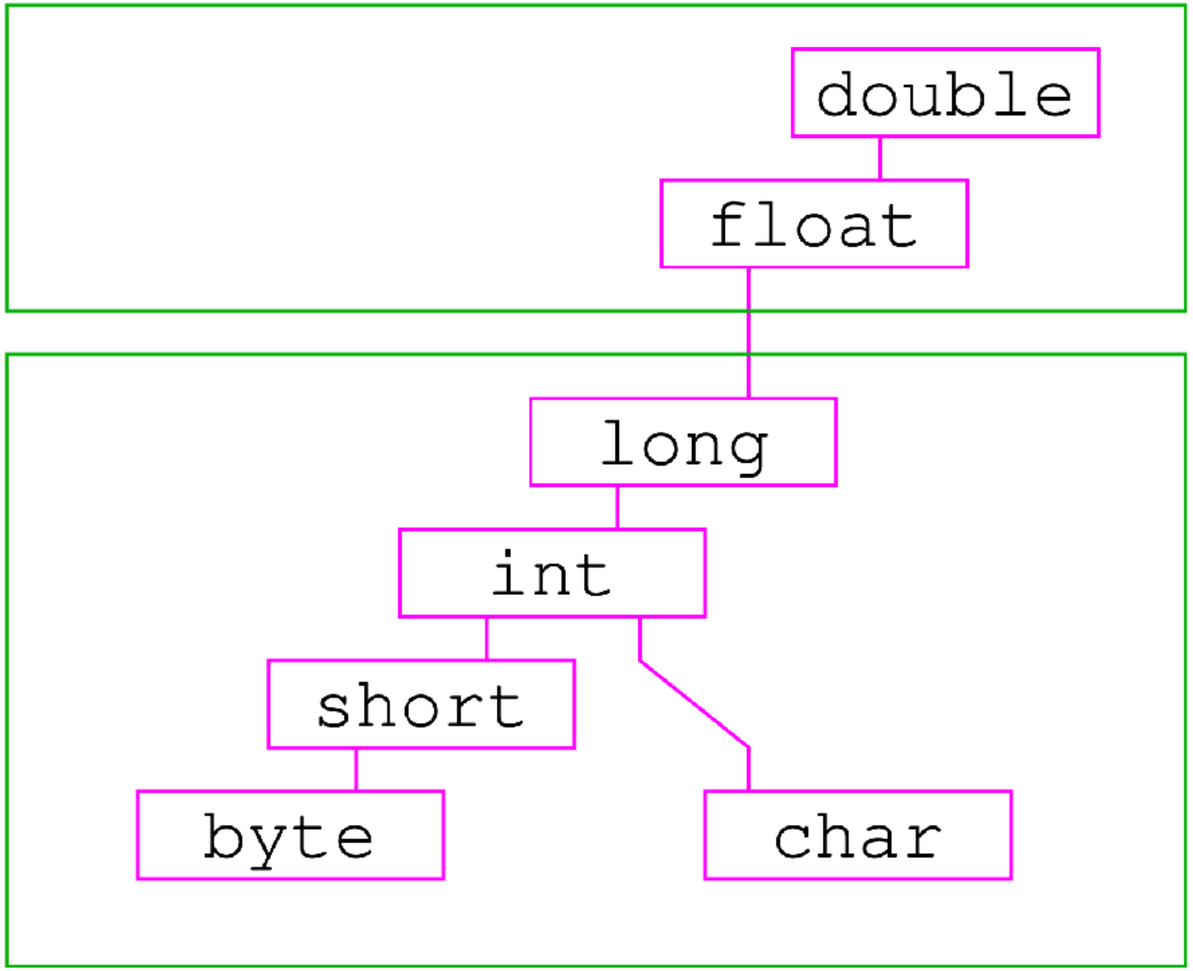
```
int i1 = 5;
int i2 = 6;
int i3 = 5;
System.out.println(i1 == i2);   // false
System.out.println(i1 == i3);   // true
```

```
Point p1 = new Point();
Point p2 = new Point();
Point p3 = p1;
System.out.println(p1 == p2);   // false
System.out.println(p1 == p3);   // true
```

Basic operations

- The operators `+`, `-`, `*`, `/` and `%` exist for each of the number types
- If they are applied to a pair of arguments of different types, the more specific one is converted to the more general one beforehand (implicit type cast)

Overview of basic types



Floating point numbers

Integers

Example

```
short xs = 1;
int x = 9999999;
System.out.println(x + xs);
```

Prints the **`int`** value `10000000`

```
float xs = 1.0f;
int x = 999999999;
System.out.println(x + xs);
```

Can output floating point numbers

Prints the **`float`** value `1.0E9`

Elementary operations

Precedence	Operator	Description
	Arithmetic and comparison operators	
1	`+x, -x`	unary plus/minus
2	`x * y, x / y, x % y`	multiplication, division, modulo
3	`x + y, x - y`	addition, subtraction
4	`x < y, x <= y, x > y, x >= y`	size comparisons
6	`x == y, x != y`	equality, inequality
	Operators on integers	
1	`~x`	Bitwise complement (NOT)
	Operators on integers and truth values	
7	`x & y`	Bitwise AND
8	`x ^ y`	Bitwise either/or (XOR)
9	`x | y`	Bitwise or (OR)
	Operators on truth values	
1	`!x`	NOT
10	`x && y`	Sequential AND
11	`x || y`	Sequential OR
	Operators on integers	
4	`x << y`	Left shift
4	`x >> y`	Right shift (sign conform)
4	`x >>> y`	Right shift (without sign)

https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

Example

```
String s0 = "";
String s1 = "Hel";
String s2 = "lo Wo";
String s3 = "rld!";
System.out.println(s0 + s1 + s2 + s3);
```

... writes Hello World! on the output

Implicit conversion to **String**

- Every value in Java has a representation as a **String**
- If the operator "+" is applied to a value of type **String** and another value x, then x is automatically converted to its **String** representation

- This is a simple way to output float or double

- Example

```
double x = -0.55e13;
System.out.println("A floating point number: " + x);
```

Output on the console: `A floating point number: -5.5E12`

Exercise: what will the following program output?

```
public class Test {
    public static void main(String[] args) {
        int x = 2, y = 5;

        int exp1 = (x * y / x);
        int exp2 = (x * (y / x));

        System.out.println(exp1);
        System.out.println(exp2);
    }
}
```

Solution

Exercise: what will the following program output?

```
public class Test {
    public static void main(String[] args) {
        int x = 10, y = 5;

        int exp1 = (y * (x / y + x / y));
        int exp2 = (y * x / y + y * x / y);

        System.out.println(exp1);
        System.out.println(exp2);
    }
}
```

Solution

Exercise: what will the following program output?

```
public class Test {
    public static void main(String[] args) {
        int x, y, z;
        x = y = z = 2;
        x += y;
        y -= z;
        z /= (x + y);
        System.out.println(x + " " + y + " " + z);
    }
}
```

Solution

Outline

- General organization of the course
- Object oriented programming
- Basic types and their operations
- **Methods**
- UML class diagrams

Methods

Methods realize the dynamic behavior of objects and define functions

```
int average(int v1, int v2) {
    return (v1 + v2) / 2;
}
```

```
int fahrenheit(int celsius) {
    return (celsius * 9) / 5 + 32;
}
```

```
double voltage(double resistance, double current) {
    return resistance * current;
}
```

Method signature

- The interface of a mathematical function consists of the name of the function, an ordered list of (formal) parameters and their types, and the return type
- A method signature in Java consists of
  - Name of the method
  - Ordered list of (formal) parameters and their types (this determines the order and number)
  - Note: in Java, the return type is not part of the method signature
- Examples

```
int fahrenheit(int celsius) {
    return (celsius * 9) / 5 + 32;
}
```

Signature

Not part of the signature

```
double fahrenheit(double celsius) {
    return (celsius * 9) / 5 + 32;
}
```

Method call

- **Example**: Ohm's law

Formal parameters

```
double v(double r, double i) {
    return r * i;
}
```

```
v: voltage
i: current
r: resistance
```

- **Call:**

```
double r1 = 3.0;
double v1 = v(r1, 2.0);
```

Actual parameters

- **Syntax:** `methodname(actualparameter)`
- **Result:** An element with the return type of the method

→ What is the value of `v1`?

Example: cylinder volume

- Mathematical notation

$$volume(r, h) = r^2 * \pi * h$$

- Java declaration

```
double circleArea(double radius) {
    return radius * radius * 3.1416;
}
```

You can also use **Math.PI**

```
double cylinderVolume(double height, double radius) {
    return height * circleArea(radius);
}
```

- **`circleArea(2.0)`** returns the value `12.5664`
- **`cylinderVolume(1.5, 2.0)`** evaluates the expression **`1.5 * circleArea(2.0)`** and returns the value `18.8496`

Constructors

- Constructors are special methods that initialize attributes after reserving memory with **`new`**
- Constructors do not define a **return type** (you do not return anything, because they will automatically create a new object and return it)
- Formal parameters can be used for initialization
- Example

```
class Rational {
    double numerator, denominator;

    Rational(double n, double d) {
        numerator = n;
        denominator = d;
    }
}
```

The **void** return type

- The void return type is used when a method does not return a value
- The method is then called procedure
- Example

```
void alarm(String message) {
    System.out.println("DANGER: " + message);
}
```

  - No **`return`** necessary
  - Procedures can still change the state of an object (i.e. have side effects)
  - Declaration

```
void methodName(formalParameterList) {
    // body
}
```

  - Usage

```
methodName(actualParameterList);
```

Current object

- Normally the method depends on the state of the current object

→ This related object must be specified when calling

- This means that within a method it must be possible to address this reference object ("itself")
- For this purpose there is the keyword **`this`**

Example: using **this** in the constructor

```
class Point {
    double x, y;
    Point(double x, double y) {
        this.x = x;
        this.y = y;
    }
}
```

Use of object state in methods

- Example: define method **`shift`** in class **`Point`**

```
class Point {
    double x, y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    void shift(double dx, double dy) {
        this.x = this.x + dx;
        this.y = this.y + dy;
    }
}
```

- Call

```
Point p = new Point(3.0, 4.0);
p.shift(2.0, 4.0);
```

- Call syntax

```
object.methodName(parameter);
```

If no object is specified → **`this`** is always used implicitly

Reference class - static methods

- A static method is independent of the state of concrete objects
- Example

```
class Weather {
    static int fahrenheit(int celsius) {
        return (celsius * 9) / 5 + 32;
    }
}
```

Question: can static methods have side effects?

- Call

```
Weather.fahrenheit(15);
```

- Syntax

```
static returnType methodName(parameter) {
    // method body
}
```

- Call syntax

```
ClassName.methodName(parameters)
```

The main method

- One static method is special: the **`main()`** method

```
class K {
    public static void main(String[] args) {
        //...
    }
}
```

Field / array

- **`main(p1, ..., pn)`** in class K will be called when you execute `java K` after compiling K
- The arguments **`p1, ..., pn`** are stored in a field called **`args`**

Covered later

Example

Playground.java

```java
public class Playground {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("First argument: " + args[0]);
        }
        else {
            System.out.println("No arguments provided");
        }
    }
}
```

Run the application with an argument

launch.json

.vscode > launch.json > ...

```java
{
    "version": "0.2.0",
    "configurations": [
        {
            "type": "java",
            "name": "App",
            "request": "launch",
            "mainClass": "itp.playground.App",
            "projectName": "app",
            "args": "'Hello World'"
        }
    ]
}
```

Create a **`launch.json`** and specify the argument here

Example: Point

Move points

```
class Point {

    double x, y;

    void shift(double dx, double dy) {
        this.x = this.x + dx;
        this.y = this.y + dy;
    }
}
```

Current coordinates

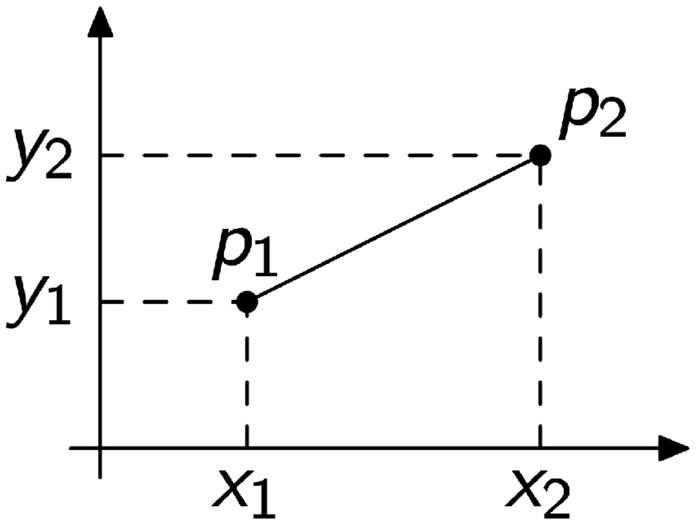
```
double getX() {
    return this.x;
}
```

```
double getY() {
    return this.y;
}
```

Move lines

Example: lines in R^2

Concept




```
class Line {
    Point p1;
    Point p2;

    Line(Point p1, Point p2) {
        this.p1 = p1;
        this.p2 = p2;
    }

    void shift(double dx, double dy) {
        this.p1.shift(dx, dy);
        this.p2.shift(dx, dy);
    }
}
```

References and method calls

- In Java, only call-by-value is used for parameter passing in method calls

- Definition of call-by-value: when calling methods, all statements in the parameters are first evaluated completely
  - Then, the values of those statements are passed as a copy to the called method

- **Note:** values of non-primitive types (i.e. objects) are references

  → A copy of the reference is passed
  - This can be used to still change the state of the referenced object

Local variables (1)

- Serve as auxiliary memory for intermediate results in methods
- Are stored on the so-called runtime stack
- Exist only as long as the method is processed

- Example (calculation of the area of a circle)

```
double circleArea(double radius) {
    double radiusSquare = radius * radius;
    return radiusSquare * 3.1416;
}
```

Local variables (2)

- In Java, **parameters** are local variables that are initialized when the method is called (call-by-value)
- Example

```
int foo(int a) {
    int x = a + 1;
    a = x * x;        // bad programming style
    return a + x;
}
```

You should **not** change input parameters in a method

```
foo(a)

int a
int x

x = a + 1;
a = x * x;
return a + x;
```

(Parameter)

Attention: simple argument types **`(int, float, ...)`** are handled differently than complex types **`(Book, Author, ...)`**

Local variables - differences to attributes

	Local variable	**Attribute**
Declaration	Within methods	Outside of methods
Lifetime	Method call	Lifetime of the associated object
Accessible	Only within a method	For all methods of the class
Purpose	Cache for values	State of the object

Outline

- General organization of the course
- Object oriented programming
- Basic types and their operations
- Methods
- **UML class diagrams**

Class diagrams: classes

- A graphical visualization of the **`Rational`** class, considering only the essential functionality, might look like this

```
┌──────────────────────────────────────────────┐
│                  Rational                    │
├──────────────────────────────────────────────┤
│ – numerator   :  int                         │
│ – denominator :  int                         │
├──────────────────────────────────────────────┤
│ + add      (y: Rational) : Rational          │
│ + equals   (y: Rational) : boolean           │
└──────────────────────────────────────────────┘
```

Class diagrams: discussion and outlook

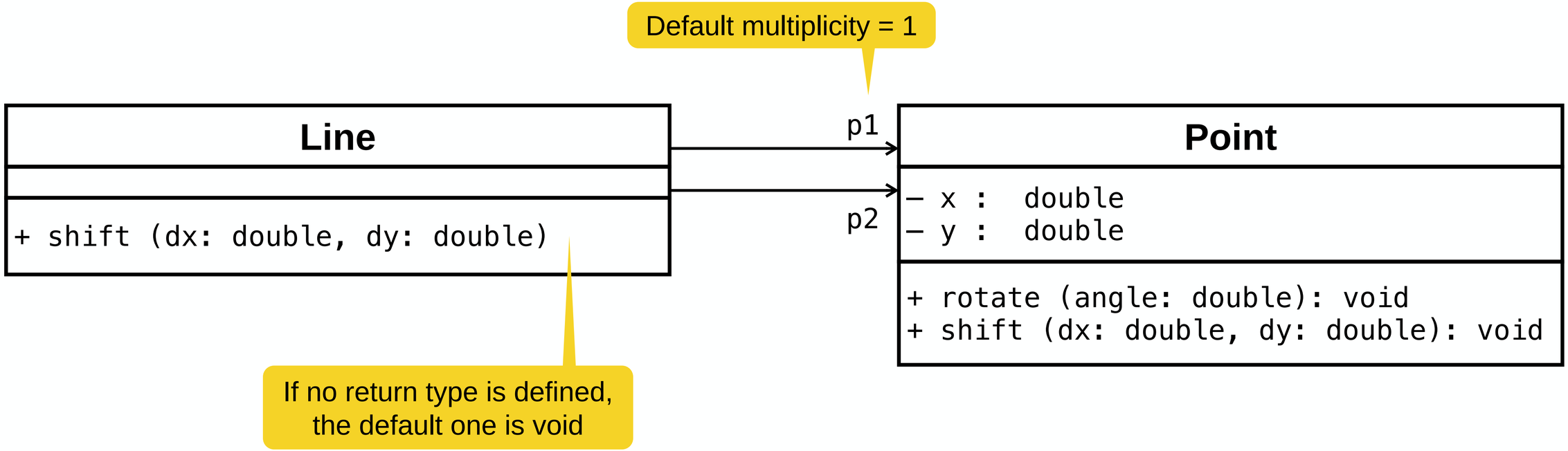
- Such diagrams are provided by UML (Unified Modeling Language), to design software systems

More on UML in the course
Introduction to Software Engineering

- For a single class, such a diagram is not really worthwhile
- However, if a system consists of many classes, it can be used to illustrate the relationships between different classes
- UML was not developed specifically for Java
  - Types are notated differently
  - Also, some ideas cannot be modeled at all or only poorly

Class diagrams: associations

- Reference attributes are often noted as references (lines between classes)
- In the example below, a line is connected to two points p1 and p2



Next steps

- Visit a tutorial group
  - T01E01 - Calculator
  - T01E02 - Autumn Leaves
- Solve the homework independently
  - H01E01 - Lectures
  - H01E02 - Take-Off in Munich
- Read the following articles to find out more about object oriented programming in Java
  - https://www.w3schools.com/java/java_oop.asp
  - https://www.w3schools.com/java/java_methods.asp

Click on **Next** several times

→ Due until **Wednesday, October 22, 13:00**

Summary

- **Object oriented programming** is the elementary thinking in Java and many other programming languages
  - **Attributes** allow to model the state of objects
  - **Constructors** initialize objects
  - **Methods** implement the logic (behavior) of objects
- Eight **basic data types**: `boolean, byte, short, int, long, float, double, char`
- **Methods** determine the behavior of objects
  - Static methods belong to classes
- **UML class diagrams** allow to visualize the structure and relationships of classes

References

- https://www.javatpoint.com/difference-between-object-and-class
- https://www.javatpoint.com/object-and-class-in-java
- https://www.javatpoint.com/method-in-java
- https://www.javatpoint.com/java-constructor
- https://www.javatpoint.com/static-keyword-in-java
- https://www.javatpoint.com/this-keyword
- https://www.javatpoint.com/java-oops-concepts
- https://www.geeksforgeeks.org/difference-between-class-and-object